

CAPITOLUL 1

Componentele de bază ale sistemelor de calcul

CUPRINS

CAPITOLUL 1	1
1.1. SISTEMUL DE CALCUL	3
1.1.1. Definiții	3
1.1.2. Aplicațiile sistemelor de calcul	3
1.1.3. Sistemul de calcul – din diferite puncte de vedere	3
1.1.4. Structura ierarhică de organizare a calculatorului	5
1.1.5. Structura funcțională a unui sistem de calcul	6
1.1.6. Performanțele unui sistem de calcul	8
1.1.7. Clasificări	8
1.1.8. Principiile von Neumann	11
1.1.9. Modelul formal al unui calculator	12
1.1.10. Evoluția sistemelor de calcul	13
1.2. SISTEMUL DE INTRARE/IEȘIRE. DISPOZITIVE PERIFERICE	18
1.2.1. Clasificări	19
1.2.2. Caracterizarea unor periferice	20
1.2.2.1. Discurile dure. Hard-diskul	20
1.2.2.2. Discurile flexibile. Discheta	21
1.2.2.3. Compact discurile	21
1.2.2.4. Monitoare (graphic display)	22
1.2.2.5. Principalele adaptoare video	24
1.2.2.6. Mouse-ul	24
1.2.2.7. Rețele de comunicare	25
1.2.2.8. Placa de rețea	27
1.2.2.9. Modemul	27
1.2.3. Canalul de intrare / ieșire	27
1.2.4. Legarea perifericelor la SC. Unitățile de interfață I/O	28
1.2.5. Transferul eficient de date. Procesoare specializate	29
1.2.6. Magistrale	30
1.2.6.1. Clasificarea magistrelor	33
1.2.6.2. Standarde de magistrală	35
1.3. UNITATEA DE MEMORIE	38
1.3.1. Structura fizică a memoriei	38
1.3.2. Tehnologii de realizare a memoriilor. Memorii semiconductoare	39
1.3.3. Structura ierarhică de organizare a memoriei	42
1.3.4. Memorii cache	45
1.3.5. Memoria virtuală	46
1.4. PROIECTAREA CALCULATOARELOR MODERNE	50

1.1. Sistemul de calcul

1.1.1. Definiții

Sistemul de calcul (SC, *Computer System* sau calculator) este un ansamblu de componente interconectate, destinat prelucrării datelor. În acest context, toate componentelor sistemului de calcul implicate în funcționarea acestuia devin **resursele sale**.

Adesea, un SC este descris prin cele două subsisteme principale de componente, anume componenta **hardware** și subsistemul **software**, la care trebuie să adăugăm **resursele informaționale**. Astfel, avem următoarele definiții și descrieri:

- totalitatea resurselor fizice formează componenta **hardware** și constă din resursele materiale, cu masă și volum, cum ar fi: procesoare, memorii, dispozitive de intrare/ieșire, ș.a. O clasificare acoperitoare a resurselor fizice poate porni de la gradul de răspândire (utilizare) a unei componente fizice a calculatorului la momentul la care se face clasificarea. Rezultă astfel: (a) resurse fizice care formează un sistem de calcul minimal, (b) resurse fizice care formează un sistem de calcul uzual și (c) resurse fizice care formează un sistem de calcul specializat.
- totalitatea resurselor logice formează subsistemul **software** și constă din resursele nemateriale, recunoscute ca aplicații sau programe. După importanța lor în SC, acestea se împart în (a) software pentru / de sistem (*systems software*) și (b) software pentru aplicații (*applications software*). Aplicațiile de sistem oferă servicii pentru uz comun, în special pentru gestionarea sistemului însuși și, eventual, a rețelei/rețelelor din care acesta face parte. Din această categorie fac parte: sistemul de operare, drivere, servere ș.a. Aplicațiile propriu-zise pot fi grupate după diverse criterii, rezultând, de exemplu: medii integrate de dezvoltare și programare, aplicații utilitare, aplicații utilizator.
- **resursele informaționale** constau din datele și fișierele de date ale utilizatorilor.

În cele ce urmează, toate considerentele legate de sistemele de calcul vor fi orientate către componenta hardware, eventual cu completări care să vizeze direct legătura acesteia cu detalii software.

1.1.2. Aplicațiile sistemelor de calcul

Pe bună dreptate se spune despre calculator că este *mașina care a revoluționat lumea*. Această afirmație poate fi susținută arătând diversitatea domeniilor în care calculatorul / procesorul este nelipsit.

1.1.3. Sistemul de calcul – din diferite puncte de vedere

Abordarea componentei hardware a sistemului de calcul poate fi făcută din cel puțin din trei puncte de vedere:

1. organizarea componentelor calculatorului;
2. construirea sistemului de calcul;
3. arhitectura sistemului de calcul.

Organizarea componentelor calculatorului se referă la cunoașterea modului în care operează *fiecare* dintre componentele hardware și la identificarea posibilităților lor de interconectare. Din această perspectivă, este importantă *performanța* fiecărei resurse fizice, prin caracteristicile proprii. Detaliind, din acest punct de vedere, se pot descrie:

- a) circuitele logice digitale care reprezintă structura internă a fiecărei componente hardware;
- b) modurile de reprezentare a datelor în memoria calculatorului;
- c) modul de operare a componentelor hardware, prin ele însele și în relație cu alte componente.

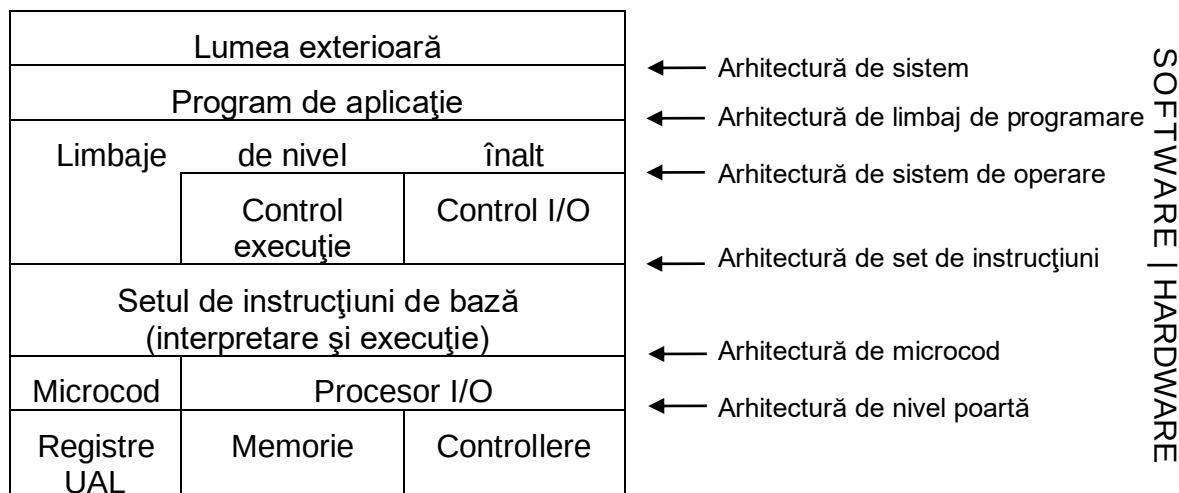
Construirea sistemului de calcul (proiectare, design) se referă la acțiunea de determinare a tipului componentelor hardware care trebuie *selectate* și a soluțiilor de interconectare a acestora, astfel încât calculatorul construit din ansamblul lor să răspundă optim cerințelor *utilizatorului* pentru care este proiectat. Concret, din acest punct de vedere, descrierea resurselor fizice se referă la:

- a) identificarea dependențelor dintre componente și a impactului lor asupra viitorului sistem;
- b) prioritizarea obținerii unui raport adecvat calitate – cantitate în utilizarea resurselor (preț, timp, memorie, etc.);
- c) optimizarea soluțiilor de transfer de conținut între componente.

Arhitectura sistemului de calcul se referă atât la aspectele de organizare a componentelor, cât și la cele de proiectare și construire a sistemului, văzut de utilizator.

Când ne referim la **arhitectura unui SC** trebuie să dăm specificațiile diferitelor unități funcționale și conectările fizice și logice dintre ele.

Conform Dicționarului enciclopedic¹, (INFORM) **arhitectura calculatoarelor** se referă la asamblarea subsistemelor funcționale într-un calculator numeric, pentru a obține performanțele impuse în condiții de cost avantajoase.



¹ <https://dexonline.ro/definitie/arhitectura/definitii>, sursa: [DE \(1993-2009\)](#)

Conform Tanenbaum [13], sintagma *arhitectura calculatoarelor* este adesea utilizată cu semnificația simplă de *organizarea și proiectarea calculatoarelor*. Detaliind din punct de vedere practic, într-un sistem de calcul se pot distinge mai multe nivele de arhitectură (vezi figura următoare), fiecare nivel fiind definit de legătura pe care o stabilește între subsistemele sistemului gazdă.

În general, prin *arhitectură de nivel* ne referim la interfața dintre două astfel de module funcționale. **Arhitectura de sistem** sau **arhitectura sistemului de calcul** este cea dintre sistemul de calcul și mediul exterior lui. La limita dintre *hardware* și *software* se află arhitectura setului de instrucțiuni care reprezintă nivelul elementar la care sunt decodificate și executate instrucțiunile recunoscute de unitatea de prelucrare.

1.1.4. Structura ierarhică de organizare a calculatorului

Din punctul de vedere al rolului fiecărei resurse a sistemului de calcul, putem privi organizarea internă a calculatoarelor moderne pe două sau mai multe nivele. Fiecare nivel este construit pe baza predecesorului său și reprezintă o abstractizare distinctă, cu obiecte și operații specifice.

În general, se pot identifica următoarele șase nivele conceptuale [Tanenbaum]:

Nivelul 0. Nivelul logic digital. La acest nivel interesează **porțile logice** care intră în structura fiecărei componente interne a calculatorului. Fiecare poartă este construită dintr-un număr mic de tranzistoare. Câteva porți pot fi combinate pentru a forma un bistabil (o memorie de 1 bit), care poate stoca 0 sau 1. Memoriile de 1 bit pot fi combinate pentru a forma regiștri. În acest curs, nivelul logic digital este subiectul capitolului al doilea.

Nivelul 1. Nivelul microarhitecturii. La acest nivel calculatorul este o colecție de **regiștri** (care formează memoria locală) și un circuit capabil să execute operații aritmetice și logice (numit **UAL**, unitate aritmetico-logică). Regiștri sunt conectați la UAL pentru a forma o cale de date (engl. *datapath*) prin care se transferă datele între regiștri și UAL. Operația de bază la nivelul căii datelor este selecția regiștrilor asupra cărora operează UAL. După felul în care este controlat transferul datelor deosebim mașini de calcul cu control microprogramat sau mașini de calcul cu control bazat și pe circuite ale componentelor hardware.

Nivelul 2. Nivelul arhitecturii setului de instrucțiuni sau nivelul ISA (engl. *Instruction Set Architecture*). Fiecare producător de calculatoare publică un manual pentru mașinile pe care le vinde, manual care se referă la acest nivel ISA, dar nu și la nivelele de dedesubt. În acest manual, producătorul descrie **limbajul mașinii** pe care o vinde, adică setul de instrucțiuni mașină recunoscute de mașina respectivă și pe care, implicit, aceasta le poate executa. Această descriere constă în detalierea instrucțiunilor executate în mod interpretat de către componenta de control (microprogram sau circuite hardware).

Nivelul 3. Nivelul mașină al sistemului de operare. La acest nivel se detaliază structura și funcțiile sistemului de operare al mașinii. Față de nivelul ISA, sunt adăugate instrucțiuni noi pentru gestionarea resurselor, există o organizare diferită a memoriei, se remarcă posibilitatea execuției paralele și/sau concurente a programelor (task-urilor).

Observație. În general, nivelele de la 1 la 3 nu sunt proiectate pentru a fi utilizate de programator obișnuit. Ele sunt un suport pentru aplicațiile dezvoltate la nivele superioare de către programatorii de aplicații. O altă diferență importantă între nivelele 1-3 pe de o parte și nivelele 4,5 pe de altă parte este natura limbajului oferit. Limbajele mașină ale nivelelor inferioare sunt numerice (programele executabile sunt secvențe greoaie de numere binare), în timp ce programele dezvoltate la nivelele superioare folosesc limbaje care conțin cuvinte uzuale sau abrevieri ușor de înțeles.

Nivelul 4. Nivelul limbajului de asamblare. Acest nivel oferă programatorilor posibilitatea de a scrie programe pentru nivelele inferioare într-o formă simbolică, mai accesibilă decât formatul impus de limbajul mașină.

Nivelul 5. Nivelul limbajului orientat pe problemă. La acest nivel sunt proiectate limbaje pe care le vor folosi programatorii de aplicații care au de rezolvat probleme specifice. Astfel de limbaje sunt uzual numite **limbaje de nivel înalt**.

Mulțimea de tipuri de date, operații și caracteristici ale fiecărui nivel se numește **arhitectura** nivelului respectiv. Cu alte cuvinte, arhitectura se referă la acele aspecte care sunt vizibile pentru utilizatorul unui nivel.

Conform Tanenbaum [7] **arhitectura calculatoarelor** se referă la studiul proiectării acelor părți ale unui sistem de calcul care sunt vizibile programatorului. În general, arhitectura calculatoarelor și organizarea calculatoarelor sunt sintagme sinonime.

1.1.5. Structura funcțională a unui sistem de calcul

Intr-un SC, resursele fizice împreună cu cele logice și informaționale sunt implicate în execuția programului curent, pentru satisfacerea cerințelor utilizatorilor în ceea ce privește: *introducerea* (recepționarea) datelor, *memorarea* (conservarea) datelor și informațiilor, *prelucrarea* informațiilor, *transmiterea* informațiilor la alte sisteme de calcul și *regăsirea* informațiilor.

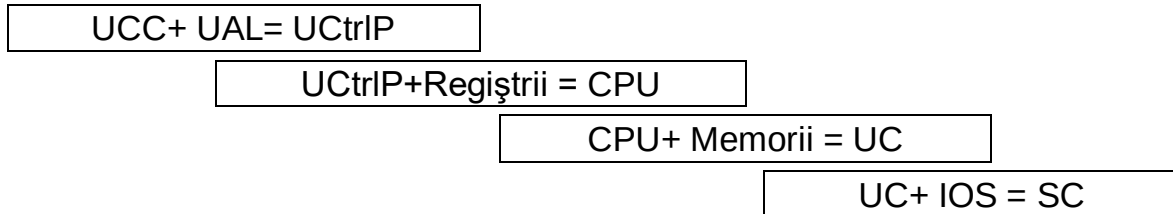
Pentru îndeplinirea acestor sarcini, resursele fizice (hardware) ale SC se grupează în **unități funcționale**, o astfel de unitate cuprinzând totalitatea componentelor cu aceeași funcție. Acestea sunt conectate atât fizic în ansamblul SC, cât și logic, prin relațiile dintre ele în timpul funcționării calculatorului.

După funcțiile pe care le îndeplinesc, unitățile funcționale se grupează în următoarele clase, enumerarea lor definind structura fizică a sistemului de calcul:

- **unități de schimb** — pentru recepționare și transmitere de conținut; unitățile de schimb formează **unitatea de comandă și control (UCC)**.
- **componentele unității aritmetico-logice (UAL)** — pentru executarea operațiilor aritmetice și logice; UAL este singura componentă a calculatorului care execută propriu-zis operații prin semnale explicite.
- **procesoare** — pentru prelucrarea datelor; totalitatea procesoarelor implicate în execuția programului curent lansat pe un SC formează **unitatea centrală de prelucrare (UCP)** a calculatorului respectiv; microprocesorul denumesc o unitate centrală de prelucrare (CPU, *Central Processing Unit*) realizată într-un singur circuit integrat; prin extindere, UCP reprezintă totalitatea componentelor de prelucrare implicate în execuția programului curent, care pot fi locale, din rețea sau în *cloud*.
- **memorii interne** — pentru stocarea datelor și instrucțiunilor la nivelul programului în execuție;

- **dispozitive periferice** — pentru preluarea, transmiterea, respectiv memorarea datelor și informațiilor externe; perifericele formează **sistemul de intrare/ieșire**.

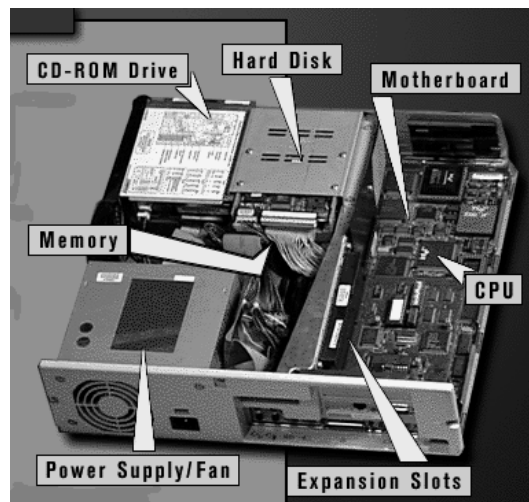
Cu aceste unități funcționale se pot scrie relații, care reprezintă structura logică a sistemului de calcul sau logica de funcționare a calculatorului. Acestea sunt:



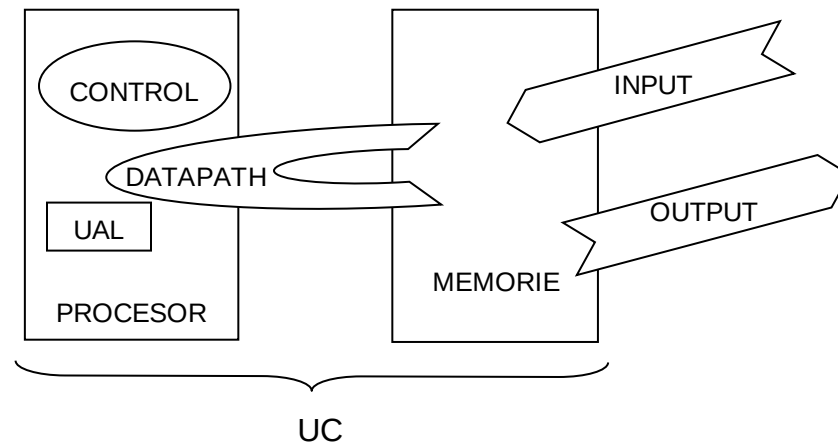
unde:

- ✓ **UCtrlP** este **unitatea de control și prelucrare**
- ✓ **UC** este **unitatea centrală**
- ✓ **IOS** este **sistemul de intrare/ieșire**.

Primele trei relații definesc *box-ul* calculatorului, în timp ce ultima relație definește legătura acestuia cu mediul exterior lui. Aceste componente se regăsesc și în figura următoare:



Orice resursă fizică a sistemului de calcul poate fi încadrată în una dintre categoriile: MEMORII, INPUT, OUTPUT, CONTROL, DATAPATH. Din figura următoare rezultă că PROCESORul preia instrucțiunile și datele din memorie, în timp ce CONTROLul trimite semnale care determină operații pe DATAPATH, în MEMORII și pe dispozitivele periferice. Datapath (calea datelor) include registre, UAL și mai multe magistrale de legătură (vezi și exemplul de transfer al datelor pe Datapath din capitolul al treilea). Cu alte cuvinte, *datapath* este acea parte a procesorului care comunică cu UAL prin intrările și ieșirile acesteia.



1.1.6. Performanțele unui sistem de calcul

Performanțele unui sistem de calcul se apreciază în funcție de:

- **viteza de execuție**, dată de numărul instrucțiunilor executate în unitatea de timp;
- **capacitatea memoriei**, dată de numărul de octeți care pot fi memorați în sistem, atât în memoria internă cât și în memoria externă, conform arhitecturii curente a sistemului;
- **timpul de acces la memorie** egal cu timpul necesar obținerii unei unități de informație memorată (octet, cuvânt): diferența de timp dintre momentul primirii datelor solicitate și momentul la care au fost solicitate;
- **fiabilitatea**, dată de numărul de defecte și/sau erori (hardware sau software) apărute în unitatea de timp;
- **tipul arhitecturii** reprezentat de caracteristicile de structură și organizare a resurselor.

1.1.7. Clasificări

Calculatoarele se pot clasifica după felul în care se memorează datele, după structura lor arhitecturală sau, o a treia variantă, după performanță (mărime, posibilități de prelucrare, preț și viteză de operare). Vom descrie în continuare grupele de calculatoare din aceste categorii.

După felul în care se memorează datele în sistemele de calcul avem:

- **calculatoare numerice** (la care ne vom referi în continuare). Datele și informațiile sunt codificate sub formă numerică binară;
- **calculatoare analogice**: sistemul de calcul este reprezentat printr-un sistem fizic în care pot fi stabilite relații matematice dinainte prescrise între variabilele continue ale sistemului fizic; spre deosebire de cazul digital, mașinile analogice nu pot fi caracterizate numai prin două stări. În general, o mașină analogică transformă o problemă matematică (teoretică sau practică) într-una bazată pe mărimi fizice (segmente, unghiuri, intensitatea curentului electric, variații de potențial) pe baza unei analogii. În final se obține un rezultat aproximativ, dar convenabil din punct de vedere practic. O mărime analogică este caracterizată de valoarea sa numerică, dar și de alte elemente specifice, de exemplu, unitatea de măsură.

Observație. În această clasificare nu ne-am referit la sistemele analogice, caracterizate de semnale sau mărimi analogice (semnale sau mărimi care variază continuu), cum este transmiterea semnalelor de telefonie.

- calculatoare hibride rezultate prin combinarea elementelor de arhitectură atât numerică, cât și analogică.

Din punct de vedere arhitectural, avem:

- sistem de calcul monoprocesor;
- sistem de calcul multicalculator;
- sistem de calcul multiprocesor.

Un sistem de calcul monoprocesor are o singură unitate centrală de prelucrare (CPU).

Un sistem de calcul multicalculator include cel puțin două unități centrale, fiecare dintre calculatoarele membre având un grad ridicat de independență.

La multicalculatoare divizarea sarcinilor de prelucrare se face cu interacțiune doar la nivelul datelor supuse prelucrării.

Modelul uzual al sistemelor multicalculator sunt rețelele de calculatoare.

Un sistem de calcul multiprocesor conține cel puțin două unități de prelucrare cu acces comun la memoria internă unică.

Procese membre ale sistemului interacționează atât la nivel hard cât și la nivel soft, dar soft-ul de bază (sistemul de operare) este unic.

Prin varianta arhitecturală multiprocesor se asigură o mărire a capacității de prelucrare prin posibilitatea de executare în paralel a unor sarcini distincte.

Totalitatea resurselor unui sistem multicalculator sau a unuia multiprocesor formează un **sistem distribuit**.

Analizate după mărime, posibilități de prelucrare, preț și viteză de operare distingem:

- supercalculatoare;
- sisteme de calcul medii-mari (*main-frame*);
- minicalculatoare;
- microcalculatoare.

Supercalculatoarele conțin mai mult de 8 UCP care pot să execute mai mult de un bilion de instrucțiuni pe secundă. De aceea, ele sunt folosite în prelucrări extrem de complexe. De obicei sunt sisteme multiprocesor.

Sistemele de calcul medii-mari (main frame) conțin mai multe UCP alături de unități principale de intrare/ieșire care operează cu viteză înaltă. Astfel, aceste calculatoare sunt apreciate în aplicațiile care solicită viteză de operare foarte ridicată.

Pentru folosirea acestor calculatoare sunt necesare instalații și proceduri de menținere în funcționare permanentă.

Exemplu. În această categorie se încadrează calculatoarele românești din familia FELIX.

Minicalculatoarele execută operații specializate. Ele sunt folosite în aplicații multiutilizator, în prelucrări de texte sau ca și componente ale mașinilor de control numeric.

Datorită facilităților oferite în cadrul proceselor de comunicații, minicalculatoarele sunt frecvent folosite în transmisii de date între sisteme dispersate geografic.

Actualmente folosirea lor este în scădere datorită utilizării microcalculatoarelor.

De obicei sunt sisteme interactive multiutilizator.

Exemple. În România s-au produs minicalculatoarele CORAL și INDEPENDENT. Sistemele VAX produse în Statele Unite sunt încă actuale prin noile lor variante.

Microcalculatoarele (PC sau Personal Computer) au apărut datorită dezvoltării tehnicii de calcul și a informaticii.

La aceste sisteme de calcul unitatea centrală este implementată cu un microprocesor.

Microcalculatoarele sunt sisteme interactive, monoutilizator.

Calculatoarele personale cunosc o dezvoltare rapidă și o diversificare continuă datorită avantajelor față de celelalte categorii, dintre care amintim:

- ✓ cunoștințele de informatică necesare unui utilizator de PC sunt minimale;
- ✓ folosirea unui astfel de calculator pentru activități zilnice se poate realiza chiar fără o pregătire de specialitate; aceasta și datorită interfețelor grafice ale sistemelor de operare și aplicațiilor care realizează legătura între utilizator și sistemul de calcul;
- ✓ au gabarit redus, pot fi chiar portabile;
- ✓ au aplicabilitate în orice domeniu de activitate.

Exemple. Microcalculatoare produse în România: aMIC, HC-85, TIM-S, CIP, HC-90, FELIX PC. În Marea Britanie s-au remarcat microcalculatoarele de tip Sinclair Spectrum, iar în Statele Unite, Commodore 64, Apple II și toate variantele de sisteme compatibile IBM PC.

Prezentăm în continuare clasificarea dată de Flynn în 1972 pentru SC. Aceasta privește calculatoarele **din punctul de vedere al mărimii fluxului instrucțiunilor și mărimii fluxului datelor**. Astfel, fluxurile pot fi:

SISD (Single Instruction Single Data Stream) pentru toate sistemele de calcul cu o unitate centrală;

SIMD (Single Instruction Multiple Data Stream) pentru unele supercalculatoare și toate SC care au o singură unitate centrală cu n unități aritmetico-logice și n module de memorie atașate unității centrale interconectate;

MISD (Multiple Instruction Single Data Stream): categorie vidă, deoarece combinația aceasta de fluxuri este inefficientă;

MIMD (Multiple Instruction Multiple Data Stream) pentru sistemele multiprocesor sau multicalculator. De exemplu, multe supercalculatoare și sistemele distribuite.

Observație. Deși Flynn a apreciat ineficientă varianta MISD, astăzi există modele paralele care se încadrează în această categorie. Este vorba despre sistemele cu execuție în *pipeline* (linie de asamblare) a sarcinilor de prelucrare și despre sistemele scalare.

În încheierea acestui paragraf dăm un tabel sintetic al **generațiilor de calculatoare**, așa cum au apărut ele cronologic (M = capacitate de memorie, V = viteză de prelucrare în instrucțiuni pe secundă).

Gen	Perioada	CPU	Memorie	Limbaje	Memorie. CPU
I	1946 – 1956	Tuburi electronice	Tambur magnetic	Limbaj de asamblare	M: 2KO; V: 1E4 i/s
II	1957 – 1963	Tranzistori	Inele de ferită	Limbaje de nivel înalt: FORTRAN, COBOL	M: 32 KO; V: 2E5 i/s
III	1964 – 1981	Circuite integrate	Memorii semicond.; Discuri magnetice	Limbaje de nivel foarte înalt (PASCAL, LISP, limbaje grafice)	M: 2 MO; V: 5E6 i/s
IV	1982 – 1989	Circuite integrate pe scară largă și foarte largă	Memorii cu bule; Discuri optice	ADA, limbaje orientate obiect	M: 8MO; V: 3E7 i/s pentru supercalculatoare
V	după 1990	Circuite integrate pe scară extrem de largă; Masini LISP și PROLOG	Arhitecturi paralele	Limbaje concurente; Limbajul natural; Limbaje funcționale (LISP); Limbaje logice (PROLOG)	V: de la 1E9 i/s la 1E12 i/s; M: suficientă pentru stocarea și prelucrarea cunostintelor (inteligenta artificială); Tehnologia vorbirii

1.1.8. Principiile von Neumann

Incepând cu 1940, după apariția lucrărilor teoretice privind calculele automate, lucrări datorate în principal lui Alan Turing, lumea cercetătorilor a devenit tot mai preocupată de construcția efectivă a unor calculatoare. Matematicianul american John von Neumann a intuit, prin analogie cu anatomia creierului uman, principiile care ar trebui să stea la baza construcției calculatoarelor.

Enumerăm aceste principii, așa cum au fost formulate la vremea respectivă.

1. Deoarece mașina este destinată calculelor, ea execută cele patru operații aritmetice. Se impune deci, existența unei componente specializate pentru calcule. (Astăzi, aceasta este numită *unitatea aritmetico-logică*.)

2. Operațiile se vor executa secvențial. Pentru a asigura elasticitatea și o relativă universalitate, se impune o distincție între instrucțiunile necesare rezolvării unei probleme particulare și controlul general asupra unor instrucțiuni executate la un moment dat. (Apare, astfel, ceea ce astăzi numim *componentă de control*.)

3. Este necesară o componentă numită *memorie internă* în care „se țin minte” pentru un timp limitat atât instrucțiunile, cât și datele necesare rezolvării problemei.

Cele trei componente, luate împreună, sunt analoage *neuronilor* din sistemul nervos central.

Se impune, continuând analogia, să existe un senzor care să recepționeze semnalele de orice fel provenite din exterior. Este necesar, de asemenea, un motor care să acționeze asupra mediului exterior. (Astăzi, acestea se numesc *dispozitive de intrare/ieșire*.)

4. Este necesară o componentă care să „țină minte permanent”, să înregistreze așa cum omul își notează ceea ce nu dorește să uite. Această componentă o vom numi *memorie permanentă*. (Astăzi, ne referim la această componentă prin *memorie externă*.)

5. Este necesară crearea posibilității de trecere de la memoria internă la cea permanentă și reciproc.

Conform acestor principii, **arhitectura von Neumann** reprezintă structura ierarhică de organizare a unui calculator construit pe baza modelului von Neumann și care conține: o unitate de memorie principală, o unitate centrală de prelucrare, o cale internă pentru transferul datelor și dispozitive de intrare - ieșire.

Cu câteva excepții, și calculatoarele actuale sunt construite pe baza acestor principii. Totuși, **sistemele paralele**, numite și arhitecturi non von Neumann, nu respectă principiul secvențialității (2), în timp ce **sistemele simbolice**, utilizate în aplicații complexe din domeniul inteligenței artificiale, se bazează pe inferențe logice, deci nu exclusiv pe calcule aritmetice (ca în principiul 1).

Opinia multor cercuri științifice este că, cele trei direcții fundamentale de cercetare care prezintă interes în această perioadă sunt: microprocesoarele, inteligența artificială și prelucrarea distribuită. Se preconizează astfel o orientare preponderent spre **arhitecturile non von Neumann**, soluții cuantice și nu exclusiv digitale binare, mai noi și mai puțin studiate...

1.1.9. Modelul formal al unui calculator

De la empirismul lui von Neumann, să trecem acum la rigoarea matematică. Încercăm să exprimăm formal sarcinile unui SC.

Notăm cu M, P, D, C cele patru componente principale: memoria, procesorul, dispozitivele periferice și, respectiv, unitatea de control.

Principalele funcții ale SC le vom nota astfel:

r – *receive* – preluarea datelor de la un dispozitiv periferic și depunerea lor într-o zonă de memorie;

t – *transmit* – este operația inversă lui *receive*;

s – *store* – preia o informație prelucrată de UCP și o depune în memorie;

f – *fetch* – preia o dată din memorie și o transmite componentei UCP spre prelucrare;

p – *processing* – prelucrarea propriu-zisă a datelor și informațiilor.

Astfel, un SC poate fi exprimat prin:

$$SC = (M, P, D, C, r, t, s, f, p),$$

unde domeniile și codomeniile celor cinci funcții sunt:

$$r : D \times C \longrightarrow M$$

$$t : M \times C \longrightarrow D$$

$$s : P \times C \longrightarrow M$$

$$f : M \times C \longrightarrow P$$

$$p : M \times P \times C \longrightarrow M \times P.$$

După cum se poate observa, fiecare funcție are, ca și componentă a domeniului de definiție unitatea de control a SC.

1.1.10. Evoluția sistemelor de calcul

„De unde... un ENIAC este echipat cu 18000 de tuburi electronice și cântărește 30 tone, înseamnă că computerele viitorului ar putea avea 1000 tuburi și, probabil, o greutate de 1,5 tone. ”

Popular Machine, martie 1949

Deși este dificil de prezis performanțele viitoarelor SC, este sigur că vor depăși cu mult caracteristicile celor actuale.

Atât creatorii de soft cât și cei de hard construiesc sistemele de calcul bazându-se pe structura ierarhică de organizare în care fiecare nivel cumulează caracteristicile nivelelor inferioare.

Tehnologiile cheie pentru procesoarele moderne sunt compilatoarele și siliconul. În timp ce siliconul permite mărirea capacităților hard, noile idei în organizarea calculatoarelor îmbunătățesc considerabil raportul preț / performanță. Dintre aceste idei, două sunt remarcabile: exploatarea paralelismului procesoarelor (de obicei prin pipeline) și exploatarea accesului direct la memorie (de obicei prin completarea cu memorie de tip cache).

J. Presper Eckert și John Mauchly de la Școala Moore a Univ. Pennsylvania construiesc prima mașină cu caracteristicile constructive și funcționale ale unui calculator electronic operațional. Această mașină, numită **ENIAC (Electronic Numerical Integrator and Calculator)** a fost finanțată de SUA și folosită în timpul celui de-al doilea război mondial, dar nu a fost făcută publică până în 1946. ENIAC era folosită în special pentru calcularea tabelului atacului de artilerie. ENIAC avea 80 picioare lungime, 8,5 picioare înălțime și câteva picioare adâncime. Fiecare dintre cei 20 regiștri de 10 biți avea 2 picioare lungime. În total, ENIAC folosea 18000 tuburi electronice. (1m = 3,2808 ft)

ENIAC executa salturi condiționate și era programabil. Programarea se făcea manual prin cuplarea (*plugging*) cablurilor și setarea comutatoarelor, iar datele erau introduse pe cartele perforate (*punched cards*). Programarea calculelor de bază dura între o jumătate de oră și o zi întreagă.

În 1944 John von Neumann a fost atras în proiectul ENIAC. Grupul vroia să îmbunătățească modalitatea de introducere a datelor și se discuta asupra memorării programelor ca o succesiune de numere. Von Neumann a contribuit la cristalizarea ideilor și a scris un memo prin care propunea un calculator cu program memorat (program-stored computer), numit **EDVAC (Electronic Discrete Variable Automatic Computer)**. Acest memo este baza a ceea ce și astăzi numim calculator von Neumann (vezi Principiile von Neumann). Mulți apreciază că această denumire acordă prea mare credit lui von Neumann și neglijează aportul (inginerilor) creatorilor Eckert și Mauchly...

În 1946 Maurice Wilkes de la Universitatea Cambridge a vizitat Scoala Moore. Întors la Cambridge, Wilkes s-a hotărât să demareze un proiect care să construiască un calculator cu program memorat, numit **EDSAC (Electronic Delay Storage Automatic Calculator)**. EDSAC a devenit funcțional în 1949 și se consideră primul calculator cu scopuri generale (engl., *general purpose calculator*), operațional și cu program memorat din lume.

În timpul celui de-al doilea război mondial au fost construite calculatoare specializate pentru decodificarea mesajelor interceptate de englezi de la nemți. O echipă de la Bletchley Park, din care făcea parte și Alan Turing, a construit în 1943 calculatorul Colossus. Această mașină a fost secretă până în 1970. După război, acest grup a avut o oarecare influență asupra pieței britanice de calculatoare.

În timp ce se lucra la ENIAC, Howard Aiken construia la Harvard un calculator electro-mecanic numit Mark-I. Acesta a fost urmat de Mark-II și apoi de două mașini cu tuburi electronice, Mark-III și Mark-IV. Față de EDSAC, care folosea o singură memorie atât pentru instrucțiuni, cât și pentru date, Mark-III și Mark-IV aveau memorii separate pentru instrucțiuni și pentru date. Aceste mașini au fost considerate împotriva calculatoarelor cu program memorat. Pentru a respecta importanța mașinilor lui Howard, astăzi, termenul de **arhitectură Harvard** descrie mașini cu o memorie principală unică, dar cu zone cache separate pentru date și instrucțiuni.

În 1947 a fost demarat la MIT (Massachusetts Institute of Technology) proiectul Whirlwind și avea ca scop realizarea unor aplicații pentru procesarea în timp real a semnalelor radar. Deși a condus la multe invenții, cea mai importantă rămâne memoria bazată pe miez (*core*) magnetic, care a fost baza sistemelor de memorare pentru următorii 30 de ani.

Firma IBM era implicată în afacerile cu cartele perforate și cu automatizarea sistemelor, dar nu a construit calculatoare până în anii 1950. Primul calculator IBM, **IBM 701**, a fost scos pe piață în 1952 și a fost vândut în 19 exemplare. Imediat după 1950 mulți erau pesimiști în legătură cu viitorul calculatoarelor, mai ales gândindu-se că piața și oportunitatea pentru aceste mașini „super specializate” trebuie să fie foarte limitată.

În 1964, după o investiție de 5 miliarde de dolari, IBM a făcut un mare pas înainte când a anunțat calculatorul **System/360**. Acest sistem însemna o îmbunătățire a tuturor performanțelor de până atunci de cel puțin 25 de ori.

Un an mai târziu, DEC (Digital Equipment Corporation) a scos **PDP-8**, primul **minicalculator** comercial, la un preț sub 20.000 \$. Minicalculatoarele au fost precursorii miniprocsoarelor: în 1971 Intel a creat primul microprocesor, **Intel 4004**.

În 1963 s-a anunțat primul **supercalculator**, nu din partea companiilor mari sau a centrelor super-tehnologizate. Seymour Cray a creat în Minnesota mașina **CDC 6600** (CDC = Control Data Corporation). Această mașină includea multe din ideile regăsite în microprocesoarele de mai târziu.

Cray va părăsi ulterior CDC pentru a forma Cray Research Inc. în Wisconsin. În 1976 Cray Research Inc. lansează Cray-1 care era în același timp și cea mai rapidă din lume, și cea mai scumpă, dar și calculatorul cu cel mai bun raport cost / performanță pentru programele științifice. În 1996 Cray Research este asimilată de Silicon Graphics.

În timp ce Cray crea cel mai scump calculator, alții se gândeau cum să folosească microprocesorul pentru a crea un calculator atât de „scump” cât să poată fi cumpărat pentru acasă. Au fost mai multe tentative de a lansa un **calculator personal**, dar remarcabil rămâne faptul că în 1977 Apple II (i.e. Steve Jobs și Steve Wozniak) a definit ce înseamnă: preț scăzut, volum mare și calitate bună pentru ceea ce avea să devină industria calculatoarelor personale.

Deși cu un avans de patru ani, Apple va sfârși pe locul al doilea... Lansat în 1981, **IBM Personal Computer** devine cel mai bine vândut calculator de orice tip. Succesul lui va da câștig de cauză lui Intel pentru piața de microprocesoare și lui Microsoft pentru sistemul de operare. Chiar și astăzi, cel mai popular CD al lui Microsoft este cel cu sistemul de operare (Windows), chiar dacă este de multe ori mai scump decât un CD cu muzică!

Producția de microprocesoare este în plină ascensiune. Iată o situație la nivelul anului 1995:

Tip	Nr de buc vândute
80x86	50.000.000
MIPS	5.500.000
Power PC	3.300.000
Sun SPARC	700.000
HP PA-RISC	300.000
DEC Alpha	200.000

Dintre cele 5.500.000 microprocesoare MIPS (*Microprocessor without Interlocked Pipelined Stages*), numai 300.000 au fost folosite în SC. Celelalte au fost incorporate pentru jocuri video, imprimante laser ș.a.

Rezumat.

În acest paragraf am definit principalele concepte necesare pentru o bună înțelegere a structurii și funcționării calculatorului. Dintre acestea, se remarcă prin importanță definiția pentru **arhitectura calculatoarelor**, pentru că acest concept dă și titlul cursului.

Partea centrală a acestui prim paragraf din curs o constituie prezentarea structurii sistemului de calcul în diferite abordări: (1) tipurile de resurse ale

calculatorului modern, (2) structura ierarhică de organizare a sistemului de calcul și (3) structura fizică și logică a sistemului de calcul.

Paragrafele următoare vor trata în detaliu tocmai componentele principale puse în evidență de structura funcțională a calculatorului, și anume: sistemul de intrare/ieșire (dispozitivele periferice) și unitatea de memorie, în capitolul întâi, structura și rolul regiștrilor și UAL în capitolul al doilea, capitolul al treilea și al patrulea fiind rezervate structurii și funcționării unităților de prelucrare, concret a procesorului.

Cuvinte cheie.

sistem de calcul
hardware, software
arhitectura calculatoarelor, nivel conceptual de arhitectură
unitate funcțională, relații logice, UAL, UCP, memorie, IOS

Verificare.

1. Care sunt principalele dumneavoastră activități curente în care folosiți calculatorul?
2. Notați-vă principalele componente (denumire și caracteristici) ale calculatorului pe care îl folosiți în mod frecvent. Decideți care sunt elementele pe care le cunoașteți, pe care le puteți descrie și care nu. Recitiți această descriere din când în când, pe măsură ce parcurgeți cursul și bifați elementele despre care tocmai ați învățat. Discutați în final cu cadrul didactic ceea ce a rămas nebifat pe lista dumneavoastră.
3. Care este legătura între structura fizică și structura logică ale sistemului de calcul?
4. Stabiliți-vă în scris câteva repere cronologice în evoluția sistemelor de calcul.

NOTIȚE

1.2. Sistemul de intrare/ieșire. Dispozitive periferice

Pentru a folosi SC, utilizatorul trebuie să introducă în calculator programele și datele aplicației sale. De asemenea, după rezolvarea tuturor sarcinilor primite, SC trebuie să înștiințeze utilizatorul asupra rezultatelor obținute.

Totalitatea resurselor SC, capabile să *organizeze și să memoreze informații externe*, pe de o parte și să *asigure schimbul de informații între SC și mediul extern*, pe de altă parte, alcătuiesc unitatea funcțională recunoscută ca **sistem de intrare/ieșire**. Concret, acesta cuprinde **dispozitivele periferice** sau, pe scurt, **perifericele**.

Principalele tipuri de dispozitive periferice legate la SC de tip IBM-PC sunt:

- **Monitorul** – prin intermediul căruia utilizatorul primește rezultatele din calculator în format text sau imagine;
- Consola (**tastatura**) – foarte comod, sub formă alfanumerică;
- **Mouse-ul** – un dispozitiv de manevrare a unui cursor grafic care permite folosirea comodă a interfeței grafice a sistemului de operare și a aplicațiilor active;
- **Imprimanta** – cu rol în transmiterea informațiilor din calculator pe suport extern, de obicei hârtie, dar și sub formă de machete 3D stratificate. La SC recente, imprimanta dispune de diferite seturi de caractere și de puternice facilități grafice, fiind integrată în soluții multifuncționale împreună cu facilități de scanare, fotocopiere, ș.a.
- **Hard-discul** și **SSD-ul** – ca principale dispozitive de memorare a informațiilor externe;
- **Unitatea de disc flexibil** – un SC poate avea una sau mai multe unități de disc flexibil. La început s-au folosit dischetele de 8". Au urmat cele de 5.25" și apoi unitățile de 3.5".
- **Unități de disc compact**, pentru citirea CD-urilor.

Pe lângă acestea, la SC se mai pot conecta și alte periferice, întâlnite mai rar în practică. Ele sunt destinate unor categorii de utilizatori specializați în anumite operații. Dintre acestea amintim: plotter, scanner, unitate ZIP, overhead, CD-writer.

Parametrii care caracterizează performanțele unui periferic sunt:

- 1) modul de acces;
- 2) timpul de acces;
- 3) rata de transfer de date;
- 4) capacitatea;
- 5) costul.

În funcție de tipul fiecărui dispozitiv periferic, unii dintre acești parametri pot să nu aibă semnificație.

1.2.1. Clasificări

Din punctul de vedere al **direcției de transfer al informației**, avem: **perifere de intrare** (*input device*) – transferă conținutul către calculator, **perifere de ieșire** (*output device*) – transferă conținutul de la calculator și **perifere de intrare/ieșire** (*I/O device*). Din această ultimă categorie fac parte și rețelele interne / plăcile de conexiune destinate comunicării între resursele fizice și/sau între perifere.

Din punct de vedere **funcțional**, dispozitivele perifere se împart în:

1) **perifere de schimb**: imprimanta, plotter-ul, videoterminalele, rețelele de comunicare cu alte perifere;

2) **perifere purtătoare de informații permanent pe medii specifice**: benzile magnetice, discurile (SSD-urile, hard-discurile, dischetele), CD-urile, stick-urile de memorie. Acestea pot fi discutate și în contextul unității de memorie, fiind suporturi de memorie permanentă.

Din punctul de vedere al **tipului de transfer al informației**, avem **perifere bloc** și **perifere caracter**.

Periferele bloc sunt caracterizate de faptul că organizează informația în blocuri de lungime fixă, fiecare bloc având propria adresă.

Un exemplu de periferic bloc bazat pe suport magnetic este hard-discul, la care unitatea logică de schimb dintre conținutul pe periferic și memoria internă este **blocul de informație**. Acesta este format din unul sau mai multe sectoare vecine, care aparțin aceleiași piste. Un bloc va conține, pe lângă succesiunea propriu-zisă de *octeți cu informație*, un număr suplimentar de biți, numiți *biți de control*, destinați verificării corectitudinii informației memorate în blocul respectiv. Pentru această completare sunt cunoscute două metode: schema de codificare polinomial-ciclică și schema cu biți de paritate încrucișată.

Periferele caracter sunt caracterizate de faptul că furnizează sau primesc un *flux de octeți*, fără nici o structură de grupare a acestora. În consecință, octeții nu sunt adresabili și fiecare octet este disponibil ca și *caracter curent* până la apariția următorului caracter în/pe flux.

Exemple: imprimanta, terminalele cu tastatură și ecran, mouse-urile.

Din punctul de vedere al **partenerului implicat în utilizarea perifericului respectiv**, avem perifere care interacționează cu omul și perifere care interacționează numai cu mașina.

O caracteristică importantă pentru aprecierea unui periferic este dată de **rata de transfer**. Aceasta reprezintă numărul de unități de informație pe care le poate transfera perifericul respectiv într-o secundă în cadrul comunicării cu procesorul, cu memoriile sau cu alte perifere.

În tabelul următor enumerăm câteva dintre cele mai folosite perifere și caracteristicile lor esențiale. Rata de transfer reprezintă cantitatea maximă de conținut (date, informații) care poate fi transferată de componenta respectivă în unitatea de timp. Această valoare este determinantă pentru performanța componentei, prin ea însăși și în legătură cu posibilitățile de sincronizare pe frecvența de tact a procesorului.

Device	Tip	Partener	Rata de transfer (KB / sec)
Tastatură	intrare	om	
Mouse	intrare	om	
Cititor de voce	intrare	om	
Scanner	intrare	om	
Redare voce	ieșire	om	
Imprimantă laser	ieșire	om	
Monitor	ieșire	om	
Modem	intrare sau ieșire	mașină	
Rețea LAN	intrare sau ieșire	mașină	
Disk flexibil	memorare	mașină	
Disk optic (CD)	memorare	mașină	
Disk magnetic	memorare	mașină	

1.2.2. Caracterizarea unor periferice

Observație. Conținutul din acest paragraf este corect, abstracție făcând de referirile la performanțele componentelor, valori care trebuie raportate la nivelul anilor 2010 - 2015.

1.2.2.1. Discurile dure. Hard-discul

Din punct de vedere fizic, un volum de disc magnetic are structura din figura următoare.

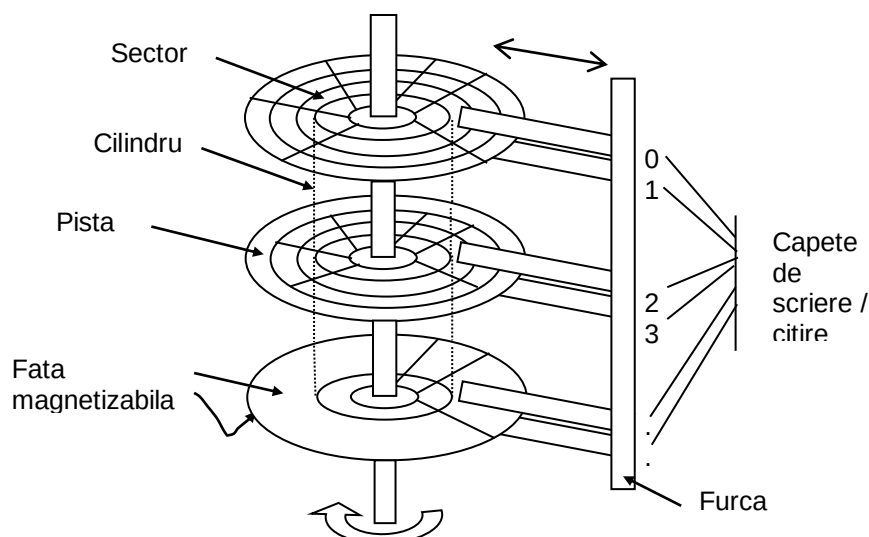
Volumul se montează pe o unitate de disc, unde *se rotește* cu o viteză constantă. Suprafața de memorare a unui disc este structurată pe trei nivele (pistă, cilindru, sector) și depinde de patru constante de construcție.

Suprafața fiecărei fețe active de memorare este divizată logic în coroane circulare concentrice numite **piste**. Numărul de piste de pe o față este prima constantă de construcție. *La o rotație completă se citește o pistă.*

În cazul în care volumul conține mai mult de două fețe active (un platan), situație frecvent întâlnită în practică, atunci fiecare față activă are același număr de piste și toate pistele cu aceeași rază formează un **cilindru**. Numărul de fețe active ale unui volum de disc este a doua constantă de construcție.

Fiecare pistă este împărțită în mai multe **sectoare**. Numărul de sectoare pe pistă este a treia constantă de construcție. În particular, organizăm conținutul pe disc în blocuri și astfel, *blocul devine unitatea de adresare a informației pe disc*.

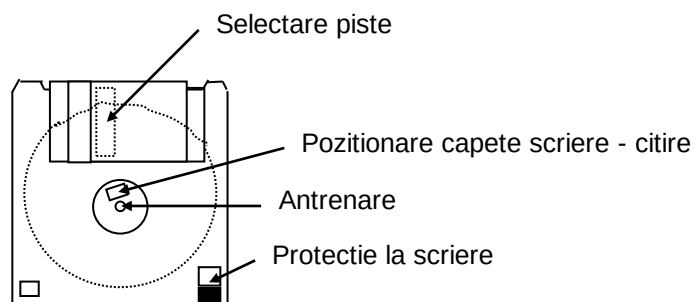
În fine, numărul de octeți dintr-un sector (bloc) este a patra constantă de construcție, de regulă, acest număr este o putere a lui 2.



Hard-discul este creat după anul 1985 și a rivalizat cu oricare dintre tipurile de discuri cunoscute înainte de SSD (engl., *Solid State Drive*). Capacitatea lor de memorare depășește capacitatea de memorare a altor discuri dure.

1.2.2.2. Discurile flexibile. Discheta

Un *disc flexibil* este format dintr-o folie magnetizabilă pe ambele fețe, îmbrăcată complet în material plastic nedeformabil (la discurile de 3.5"). Principial, îmbrăcămintea are prevăzute patru obturații (vezi figura următoare): una centrală pentru antrenare, una radială pentru selectarea pistelor și un orificiu în apropierea celui de antrenare, destinat poziționării capetelor de scriere - citire.



Al patrulea orificiu este în partea dreaptă jos: dischetele de 3.5" dispun (în vederea protecției la scriere) de un comutator cu două poziții: protejat la scriere, respectiv neprotejat. Astfel, dacă orificiul este închis atunci este permisă scrierea pe dischetă, iar dacă orificiul este deschis, se interzice scrierea.

Pentru o dischetă de 3.5" formatată la 1.44MB constantele de construcție sunt: 80 piste, 18 sectoare pe pistă, 2 capete de scriere-citire, 300 rotații pe minut, rată de transfer de 500Kbps (kilobiți pe secundă).

1.2.2.3. Compact discurile

Pentru gestionarea unor baze de date mari, casele de software utilizează din ce în ce mai mult discuri compacte, numite CD-ROM, ale căror capacități depășesc

1000 MO. În general, informațiile de pe CD-ROM-uri sunt de tip *read-only* și conțin de regulă baze mari de date cu caracter de consultare publică. Pe lângă CD-urile *read-only*, încep să se răspândească pe piață și CD-urile reinscriptibile.

Primele astfel de discuri conțineau baze de date documentare pe diverse domenii științifice: informatică, electronică, matematică, enciclopedii, etc.

1.2.2.4. Monitoare (graphic display)

Orice SC are un monitor (ecran) prin care comunică cu utilizatorii.

Funcționarea monitorului se bazează pe tehnologia de realizare a televizoarelor, adică este folosit un tub catodic (CRT - cathode ray tube). Astfel, un fascicul de raze scanează imaginea linie cu linie, de 30 - 75 ori pe secundă. La această rată de scanare utilizatorul nu poate observa raza pe ecran.

Imaginea poate fi interpretată ca o matrice de elemente luminoase elementare (*picture elements*), numite **pixeli**. De aceea, imaginea poate fi reprezentată printr-o matrice de biți, numită **bit map**. Dimensiunile acestei matrice dau **rezoluția ecranului**. Cele mai simple monitoare folosesc câte un bit pentru reprezentarea unui pixel și, astfel, pot reda numai imagini alb-negru. Urmează monitoarele care folosesc 8 biți pentru un pixel, care pot reprezenta astfel 256 nuanțe de gri ($256 = 2^8$). În fine, monitoarele color folosesc 24 biți pentru reprezentarea unui pixel și 8 biți pentru fiecare dintre culorile fundamentale, adică roșu, verde și albastru (RGB - Red, Green, Blue).

În particular, calculatoarele portabile au ecrane cu cristale lichide (LCD - liquid crystal display). Diferența esențială este faptul că un pixel LCD nu este o sursă de lumină.

Indiferent de tipul de monitor, componenta hardware pentru grafică conține mai multe buffere de imagine (*raster refresh buffer* sau *frame buffer*) pentru a memora imaginea sub formă de *bit map*. Imaginea de reprezentat pe ecran este memorată în *frame buffer* și, conform cu rata de actualizare a imaginii (*refresh rate*), fiecare bit al fiecărui pixel este citit, interpretat și reafășat pe ecran conform cu noile caracteristici (valori). Scopul realizării unui *bit map* este de a reprezenta fidel imaginea.

Competiția între sistemele grafice se dă pentru că ochiul uman detectează foarte bine fiecare schimbare de pe ecran. De exemplu, este neplăcut să se observe diferența dintre porțiunea de ecran pe care imaginea a fost actualizată, față de cea neactualizată.

Un monitor poate lucra în **regim text** sau în **regim grafic**.

Pentru conectarea monitorului la calculator se folosesc două modalități: legarea serială sau legarea prin adaptor video.

A. Legarea serială

Standardizat sub numele RS-232, acest mod de legare permite atât legarea locală cât și legarea la distanță (geografic vorbind).

Transmisia se efectuează bit după bit. Pentru fiecare bit transmis se verifică un așa-numit *protocol de comunicație*, în scopul asigurării corectitudinii transmiterii. Viteza de transmisie variază de la 300 bps (biți pe secundă) la 19.200 bps.

Atât tastatura, cât și ecranul se supun acestui mod de conectare.

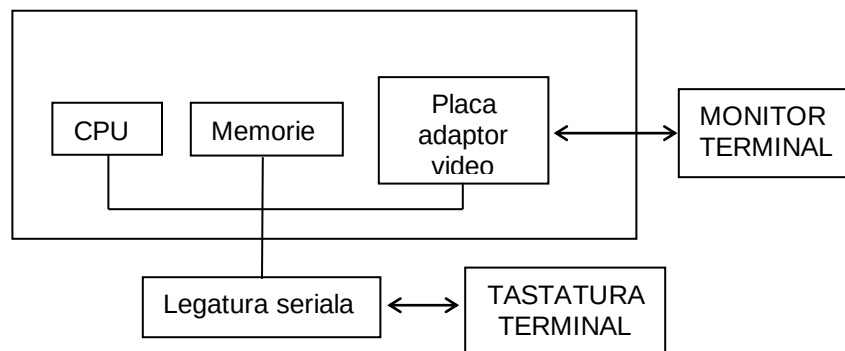
B. Legarea prin adaptor video

A doua mare categorie de monitoare sunt cele pentru care ecranul este legat la memoria calculatorului printr-o placă numită **adaptor video**.

Acest adaptor înglobează în el un **video-controller** care generează continuu un flux de electroni. La atingerea ecranului, aceștia se transformă în puncte luminoase. De aceea, la legarea prin adaptor video, ecranul este parcurs permanent linie cu linie (fenomenul se numește **baleiere ecran**), activându-se fiecare punct luminos în conformitate cu informațiile existente într-o porțiune a memoriei operative numită **memoria ecran**. Actualizarea ecranului în această manieră se desfășoară în ritmul de 45-70 baleieri pe secundă.

Spre deosebire de legarea serială, aici o modificare a imaginii este un efect al modificării memoriei ecran. Acest lucru elimină necesitatea verificării prin protocoale a corectitudinii datelor, ceea ce face să crească mult viteza de afișare.

Acest sistem de lucru, ilustrat în figura următoare, este cunoscut și sub numele de **legarea terminalelor cu memoria mapată**.



Ecranul poate lucra, dar nu simultan, fie în regim text, fie în regim grafic. În regim text vede (de regulă) 25 de linii a câte 80 de caractere fiecare, iar în modul grafic o matrice de pixeli, ale cărei dimensiuni depind de caracteristicile adaptorului video.

Evident că, pentru modul de lucru grafic memoria ecran este mult mai mare decât cea necesară pentru modul de lucru text.

În memoria ecran, fiecărui caracter (în mod de lucru text) sau fiecărui pixel (în mod de lucru grafic), îi sunt atașate așa numitele **atribute ale imaginii**. Aceste atribute se referă la:

- ✓ **strălucirea** punctului luminos (*brightness*);
- ✓ la posibilitatea imaginii de a **pulsa** pe ecran (*flashing*);
- ✓ la posibilitatea imaginii de a apărea în **video invers** (adică situația în care se schimbă culoarea fondului cu cea a punctelor luminoase);
- ✓ la **culoarea** pixelului sau a caracterului.

1.2.2.5. Principalele adaptoare video

Primul adaptor video disponibil pentru IBM-PC a fost unul pentru ecran monocrom. El s-a numit **MDA** (*Monochrom Display Adapter*) și a fost creat în 1981. Caracteristici: nu oferă facilități grafice, poate să afișeze numai caracterele ASCII standard și cele speciale din setul IBM.

Adaptorul **Hercules**, prescurtat **HGC**, (*Hercules Graphics Card*) a fost creat de firma cu același nume în 1982. HGC permite atât afișare alfanumerică cât și afișare grafică. Tehnicienii de la Hercules au rezolvat cu această ocazie îmbinarea textului cu grafica, iar standardul firmei s-a impus rapid pe piață. HGC permite comutarea între modul de lucru impus de MDA și modul grafic cu o rezoluție înaltă: 720*348 pixeli.

Adaptorul **CGA** (*Color Graphics Adapter*) a început să fie livrat în 1981. Pe lângă modul de lucru alfanumeric (80*25 caractere), acesta permite un mod de lucru cu patru culori și o rezoluție de 320*200 pixeli. Calitatea afișajului nu este deosebită, datorită rezoluției destul de reduse, în schimb se pot folosi culorile.

Placa adaptor **EGA** (*Enhanced Graphics Adapter*) a apărut pe piață la începutul anului 1985. Ea moștenește modurile de lucru de la MDA și CGA, dar oferă posibilități grafice mult mai mari. Astfel, pot fi utilizate simultan 16 culori din 64 posibile, cu o rezoluție de 640*350 pixeli. Evident, calitatea imaginii este mult mai bună decât cea oferită de adaptoarele precedente.

Interfața **VGA** (*Video Graphics Array*), apărută în 1987, aparține unui nou standard grafic. Este compatibilă cu plăcile anterioare, dar oferă posibilități mult sporite față de acestea. În mod grafic are o rezoluție de 640*480 pixeli cu 16 culori simultane din 262.144 posibile, sau 256 culori simultane cu 320*200 pixeli. În modul text caracterele sunt formate din 144 de pixeli (matrice de 9*16 pixeli pe caracter), față de 64 la CGA, 112 la EGA, 126 la MDA și HGC.

În momentul de față există mai multe variante îmbunătățite de plăci VGA, cunoscute generic sub numele de **SVGA** (*Super VGA*).

1.2.2.6. Mouse-ul

Prin "mouse" ne referim la perifericul de pointare (*pointing device*) inventat în 1967 și care a intrat în componența unui sistem de calcul de prin anii '80.

Principiul de funcționare se bazează pe utilizarea unei bile montată astfel încât să fie în contact cu doi cilindri de dimensiuni foarte mici. Unul dintre acești cilindri corespunde axei Ox, adică direcției orizontale, în timp ce celălalt corespunde axei Oy, adică direcției verticale.

Cilindrii sunt manevrați mecanic sau (la modelele moderne) antrenează câte o roțiță dințată prin intermediul căreia un LED (Light Emitting Diode) luminează un senzor foto.

Mișcarea fizică a mouse-ului pe *pad* determină învârtirea bilei, care determină rotirea cilindrilor Ox și Oy (după cum mouse-ul este mișcat orizontal, vertical sau pe diagonală) și, în continuare, actualizarea prin incrementare și/sau decrementare a unor contoare sistem. Aceste contoare (numărătoare) au rolul de a înregistra cât de departe a fost mișcat mouse-ul și în ce direcție.

Interfața între mouse și sistem este asigurată prin una dintre variantele:

1. mișcarea mouse-ului generează impulsuri folosind LED-ul (vezi anterior) sau
2. mișcarea mouse-ului generează operații de actualizare a valorilor unor contoare.

Periodic, procesorul semnalează impulsurile sau citește valorile contoarelor și determină deplasarea relativă a mouse-ului de la citirea anterioară. Conform cu rezultatul obținut, reprezentarea mouse-ului pe ecran este mutată proporțional. Mișcarea reprezentării mouse-ului pe ecran este lentă deoarece variația poziției mouse-ului este mai mică decât rata de citire și interpretare a procesorului.

Mouse-ul este prevăzut cu două sau trei butoane. Operația esențială în lucrul cu mouse-ul este cea de **eliberare** a unuia dintre butoane.

Legătura dintre valorile contoarelor, poziția (starea) butoanelor și poziția reprezentării mouse-ului pe ecran se face prin mecanisme software. De obicei, aceste programe sunt grupate în *driver*-ul de mouse și se instalează o dată cu perifericul. Deoarece există acest soft, utilizatorul poate seta diverși parametri de lucru cu mouse-ul: intervalul de timp caracteristic unei operații *double-click*, viteza de deplasare a reprezentării mouse-ului pe ecran, s.a.

Pe de altă parte, interpretarea permanentă a poziției mouse-ului pe ecran asigură că niciodată mouse-ul „nu va sări din ecran”. Metoda prin care sistemul primește informații despre mouse prin citirea și interpretarea semnalelor de la mouse se numește *polling* [poulin].

1.2.2.7. Rețele de comunicare

Dacă luăm în discuție comunicarea prin rețele atunci distingem două tipuri de rețele de comunicare: rețeaua internă a sistemului de calcul și rețelele de comunicare între sisteme de calcul.

Rețeaua internă a unui SC interconectează componentele fizice interne ale sistemului, concret, procesorul la memorii și la dispozitivele periferice.

Rețelele de comunicare între mai multe SC rezultă în urma interconectării componentelor a diferite calculatoare (sisteme multicalculator). Principalele avantaje ale realizării și utilizării unei rețele de calculatoare sunt: comunicarea rapidă și facilă, posibilitatea partajării resurselor, accesul la distanță.

O rețea de calculatoare se caracterizează prin:

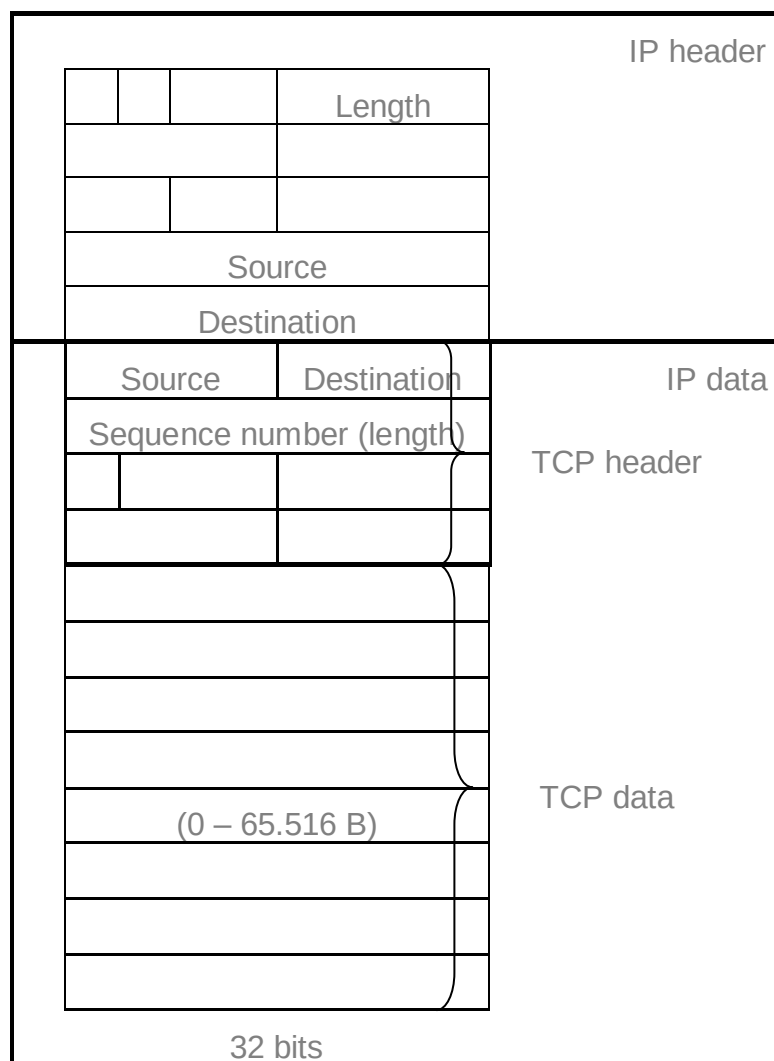
- ✓ distanța între SC conectate: de la 10 m la mii de km;
- ✓ viteza de transfer a mesajelor: de la 1 Byte pe secundă (Bps) la sute de MBps;
- ✓ topologia rețelei (organizarea SC conectate): bus, ring, star, tree etc.
- ✓ liniile de comunicare: fără linii de comunicare (rețele de tip point-to-point) sau rețele cu partajarea liniilor de comunicare (rețele multidrop).

Modele de rețele:

- 1) LAN (Local Area Network), cu exemplul cel mai popular, rețeaua **Ethernet**. Aceasta poate acoperi maxim 1 km, deci este potrivită pentru conectarea SC de la un etaj de clădire. Acceptă o viteză de transfer de 10Mbps (Megabiți pe secundă), fără control central. Mesajele transmise printr-o rețea Ethernet (pachetele) sunt dirijate de-a lungul rețelei în blocuri (64B/bloc – 1518B/bloc). Variantele moderne ale modelului LAN–Ethernet sunt Fast Ethernet (transferă 100Mbps) și Gigabit Ethernet.

- 2) Long-haul Network, cu varianta cea mai populară, **ARPANET** (Advanced Research Projects Agency of US Government). Modelul ARPANET acoperă între 10 și 10.000 km, la un transfer de 56 Kb / sec. Este de tip point-to-point. Folosește linii dedicate, închiriate de la companiile de comunicații.

ARPANET a fost precursor pentru **Internet**. Prin realizarea Internet s-a creat posibilitatea interconectării mai multor rețele de calculatoare. Mai mult, s-a standardizat această conectare la o singură familie de protocoale, și anume TCP/IP (Transmission Control Protocol / Internet Protocol). Componenta IP a acestor protocoale furnizează posibilitatea de adresare între două gazde (*host-uri*) din Internet, dar *nu* garantează securitatea și siguranța transmisiilor. Partea TCP înseamnă un protocol care poate garanta că toate pachetele sunt primite și că nu există erori de transmitere.



Aceste două protocoale (IP și TCP) lucrează împreună și formează o stivă de protocoale (*protocol stack*) în care pachetele TCP sunt încapsulate în pachetele IP. Pentru a permite comunicarea între mai multe rețele de caracteristici diferite, TCP/IP definește un format standard de pachet: un pachet IP (care conține informațiile de

adresare IP) încapsulează un pachet TCP (care conține atât informații de adresare interpretate de gazdă, cât și datele care se comunică).

1.2.2.8. Placa de rețea

Placa de rețea este componenta fizică a unui sistem de calcul prin care acesta se poate conecta la o rețea de calculatoare. O placă de rețea funcționează ca interfață fizică între calculator și cablul de rețea. Fiecare calculator din rețea (stație) și server-ul rețelei vor avea instalată câte o placă de rețea (fizic, într-un slot pe placa de bază sau incorporată direct în placa de bază). Legătura fizică între calculator și restul rețelei se stabilește după ce cablul de rețea se conectează la portul plăcii de rețea.

Fiecare „participare” a calculatorului în rețea presupune un transfer de date între calculator și placa de rețea, deci un proces de comunicare. În cazul plăcilor cu acces direct la memorie, **DMA Board** (**D**irect **M**emory **A**ccess), calculatorul-stație alocă o parte din spațiul său de memorie pentru placa de rețea.

1.2.2.9. Modemul

Atunci când se pune problema ca două calculatoare să comunice prin intermediul unei linii telefonice este nevoie și de un dispozitiv numit **modem**. Necesitatea modemului rezultă din faptul că liniile telefonice pot transporta numai semnale analogice (sunet), în timp ce calculatoarele comunică prin impulsuri digitale (semnale electronice). Un semnal digital este unul discret, care are una din două valori posibile: 0 sau 1. Un semnal analogic poate fi reprezentat printr-o curbă continuă, având un domeniu infinit de valori.

Astfel, modemul apare ca un **modulator-demodulator** de semnale, care convertește semnalele digitale în semnale analogice și invers.

Un modem include o interfață de comunicare serială (RS-232, vezi anterior *Legarea serială*) și o interfață pentru linia telefonică, RJ-11 (adică o priză de telefon cu patru fire).

Modemurile sunt disponibile atât ca modele interne, cât și externe. Un **modem intern** este instalat într-un slot de extensie al calculatorului, la fel ca orice altă placă de interfață. Un **modem extern** este ca o cutie conectată la calculator prin intermediul unui cablu serial (RS-232), care face legătura între portul serial al calculatorului și conectorul de interfață serială al modemului. În plus, indiferent de model, modemul folosește și un cablu cu conector RJ-11C pentru a se conecta la priza telefonică de perete.

1.2.3. Canalul de intrare / ieșire

Dezvoltarea arhitecturii calculatorului a condus la apariția, începând cu generația a doua de calculatoare a *canalului de intrare / ieșire*. Acesta este un procesor specializat pe operații de I/O care poate funcționa în paralel cu CPU. Pentru a fi lansat în execuție, canalul primește de la CPU o comandă de efectuare a unei operații de I/O. După lansare, cele două procesoare își continuă activitatea în paralel. La sfârșitul operației de I/O, canalul anunță acest fapt CPU prin intermediul unei *întreruperi*.

În prezent, SC folosesc două tipuri de canale, anume selector și multiplexor.

Canalele de tip selector realizează schimbul dintre memorie și dispozitivele rapide (discuri). Acestea lucrează la un moment dat cu un *singur periferic*.

Canalele multiplexor lucrează simultan cu *mai multe periferice*. Astfel, prin multiplexare, se decide, pentru fiecare octet schimbat, care este traseul acestuia între memorie și perifericele legate la canal.

1.2.4. Legarea perifericelor la SC. Unitățile de interfață I/O

Perifericele conectate la un SC au nevoie de linii speciale de comunicare pentru a se lega la procesor. Concret, aceasta se realizează prin componente hardware numite **unități de interfață** cu rol în supervizarea și sincronizarea tuturor transferurilor de intrare / ieșire.

Principalele funcții ale unităților de interfață sunt:

- 1) conversia semnalelor specifice modului de operare pe suporturi magnetice în semnale de lucru pe circuite electronice, adică de pe periferic magnetic în CPU;
- 2) sincronizarea ratelor de transfer de date de la viteza de lucru pe periferic la viteza de lucru în CPU;
- 3) decodificarea/codificarea datelor de pe periferic în format specific în CPU;
- 4) delimitarea semnalelor unui periferic de ale altora dintre cele conectate la același SC.

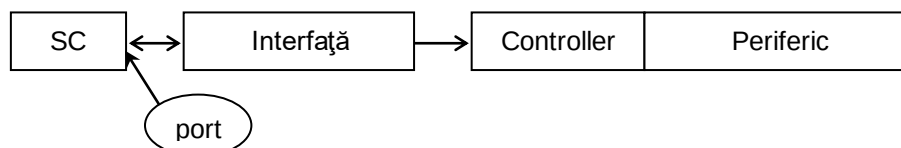
În funcție de numărul semnalelor de date prin care se realizează transmiterea informației dinspre periferic distingem **interfețe seriale** și **interfețe paralele**.

Comunicarea serială presupune că informația circulă într-o structură secvențială de biți în care alternează biții de informație cu biții de control. Numărul și semnificația biților de control sunt conform cu protocolul de comunicare prin care se asigură transmiterea datelor. În general, avem un bit de start, unul sau doi biți de stop și un bit de paritate. În **comunicarea paralelă**, informația circulă în blocuri de 8 biți (bytes), eventual, transferându-se conținutul a mai mulți octeți simultan, conform lărgimii de bandă a magistralei de comunicare folosite. Avantajul comunicării paralele constă în mărirea vitezei de transfer și în eliminarea biților de control.

În plus, comunicarea, respectiv transmiterea datelor, poate fi cu sincronizare sau **sincronă**, dacă semnalele de date sunt gestionate prin intermediul semnalelor de tact (sau de ceas). În acest caz, viteza de transmisie a datelor depinde de frecvența de tact. Dacă nu sunt implicate semnalele de tact, atunci comunicarea este **asincronă** și dirijarea datelor se poate realiza prin însăși structura datelor transmise, de exemplu, comunicarea bazată pe caracter sau comunicarea bazată pe mesaj.

Pe lângă interfață, în arhitectura componentelor moderne, fiecare periferic are o unitate de control proprie, numită **controller**, cu rol în supervizarea operațiilor specifice mecanismului de funcționare a perifericului respectiv. De exemplu, controllerul propriu imprimantei gestionează deplasarea hârtiei, timpul de listare, selectarea tipului de caracter de imprimat ș.a..

Controller-ul poate fi independent sau poate fi integrat fizic cu perifericul.



Punctul în care se conectează fizic interfața la SC este un registru de date care se numește **port**. Practic, portul este registrul prin care se realizează schimbul de informații între sistem și exterior. În funcție de tipul de interfață, acesta poate fi **port de intrare**, **port de ieșire** sau **port de intrare/ieșire**.

Portul de intrare/ieșire specifică un canal prin care informația circulă între dispozitivele hardware și procesor. Portul este identificat de procesor printr-o adresă. Astfel, fiecare resursă hardware din sistem trebuie să aibă o altă adresă de port I/O.

Concret, comunicarea între procesor și diferite periferice se face prin **magistrala de intrare/ieșire** (I/O Bus). Cuplarea perifericelor la sistemul de calcul presupune, pe de o parte, adaptarea semnalelor specifice fiecărui echipament la semnalele de pe magistrală și, pe de altă parte, reglarea fluxului de date între calculator și periferice.

Din punct de vedere logic, legătura între dispozitivele periferice, memorii și CPU este asigurată de o rutină de interfață care se va numi **driver**. Concret, un driver este o aplicație software care identifică perifericul în sistem.

Comunicarea între un periferic și CPU este permanentă. În acest sens, procesorul citește și interpretează periodic anumite date despre starea fiecărui periferic. Subsistemul de gestiune a perifericelor poate testa periodic fiecare periferic la același interval de timp sau pe baza unui sistem de priorități. Tehnica *sondării* periodice de către procesor a stării perifericelor se numește **polling** [poulin]. Testarea pe baza unui sistem de priorități este o metodă *de încredere* și poate fi cea mai potrivită soluție în sistemele mici. Tehnica *polling* este o metodă nerecomandată în sistemele puternic dependente de timp (*time-critical*) deoarece un eveniment important poate apărea imediat ce perifericul respectiv tocmai a fost testat; în acest caz evenimentul este lăsat în așteptare până la următoarea testare, ceea ce nu este convenabil. O alternativă recomandată, deosebit de utilă în toate tipurile de sisteme, este folosirea *întreruperilor*.

1.2.5. Transferul eficient de date. Procesoare specializate

Funcția principală a unei interfețe de intrare / ieșire este transferul de date. În funcție de gradul de implicare a procesorului în controlul transferului de date, avem:

1. transfer prin **program** – în care UCP se implică total în efectuarea transferului. Un astfel de program de control al transferului este o rutină de intrare / ieșire. Pe parcursul transferului de date procesorul trebuie să se adapteze la viteza de lucru a perifericului (în general mult mai mică decât a procesorului);
2. transfer prin **întreruperi** – în care UCP poate să folosească timpul liber dintre două transferuri consecutive pentru executarea altor secvențe de program (execuție concurentă); tratarea întreruperilor va fi reluată în capitolul al treilea;
3. transfer prin **acces direct la memorie** (DMA, engl. *Direct Memory Access*), fără intervenția efectivă a procesorului; această metodă de transfer necesită un circuit specializat (numit controller DMA) care are posibilitatea de a prelua controlul magistralei sistem în vederea efectuării transferului propriu-zis de date;
4. transfer prin **procesor de intrare/ieșire** – care conferă procesorului cel mai mare grad de libertate deoarece sarcinile legate de transfer sunt preluate de către o altă unitate de tip procesor, numită procesor de intrare / ieșire.

Alegerea modului optim de transfer pentru o anumită interfață depinde de o serie de factori, printre care: viteza de transfer a perifericului la care se conectează, complexitatea perifericului respectiv, restricțiile de cost și timp de realizare a interfeței respective. Implicit, structura și modul de funcționare a interfeței depind de modul de transfer adoptat.

Situația descrisă în paragrafele anterioare, în care fiecare interfață comunică cu CPU, poate fi înlocuită cu aceea în care un SC are incorporate unul sau mai multe procesoare externe cărora le încredințează sarcina de a comunica direct cu dispozitivele periferice atașate sistemului. Un astfel de procesor se va numi **procesor de intrare/ieșire, IOP**.

Ca un caz particular, un procesor care comunică cu terminale aflate la distanță (*remote terminals*) prin telefon sau alte medii de comunicare serială se va numi **DCP, data communication processor**. El este specializat pentru distribuirea și colectarea datelor direct de pe rețele de comunicații. O rețea de comunicație este compusă din dispozitive periferice de diferite tipuri: imprimante, videoterminale interactive, senzori digitali și/sau unități de intermediere a comunicării la distanță.

Folosind un DCP, SC poate rezolva cereri diferite din rețea de o manieră partajată, astfel că se poate comporta ca și când ar deservi mai mulți utilizatori simultan. Aceasta este maniera în care SC este capabil să opereze eficient în *timp partajat* (*time - sharing*).

Principala diferență între IOP și DCP constă în felul în care procesorul comunică cu perifericele. Un IOP comunică cu dispozitivele I/O printr-o magistrală I/O comună. Astfel, toate perifericele partajează magistrala comună și o folosesc pentru transferul de date înspre/dinspre IOP. Un DCP comunică cu fiecare periferic în parte printr-o singură pereche de conductori (*a single pair of wires*). Atât datele cât și informațiile de control sunt transferate de o manieră serială, astfel că rata de transfer este mult încetinită. Sarcina unui DCP este să transmită și să colecteze informații digitale înspre/ dinspre fiecare terminal. Totodată, trebuie să determine dacă informația este de tip dată sau este informație de control și să răspundă tuturor cererilor stabilite anterior prin proceduri de funcționare.

Terminalele aflate la distanță sunt conectate la DCP prin linii telefonice sau prin alte modalități de comunicare publice sau închiriate. Deoarece liniile telefonice au fost la origine destinate comunicării prin voce și SC comunică prin semnale digitale, trebuie folosită o modalitate de conversie între cele două tipuri de semnale. Aceste convertoare se numesc: *data sets, acoustic couplers* sau *modems*.

1.2.6. Magistrale

Conform cu arhitectura von Neumann a sistemelor de calcul, comunicarea între componente se realizează prin legături dedicate între perechi de componente. Aceasta este o abordare rigidă, care limitează scalabilitatea sistemului. Alternativa pentru această situație este soluția propusă de firma DEC (*Digital Equipment Corporation*) care, la sfârșitul anilor '60, a lansat pe piață primul calculator (și anume PDP 11) construit *în jurul* unei magistrale unice, magistrala UniBus.

Conceptual, o **magistrală** este un mediu comun de comunicare între componentele sistemului de calcul. Din punct de vedere fizic, o **magistrală** este un set de linii de semnal (date, adrese, comenzi, control, întreruperi, tact) care

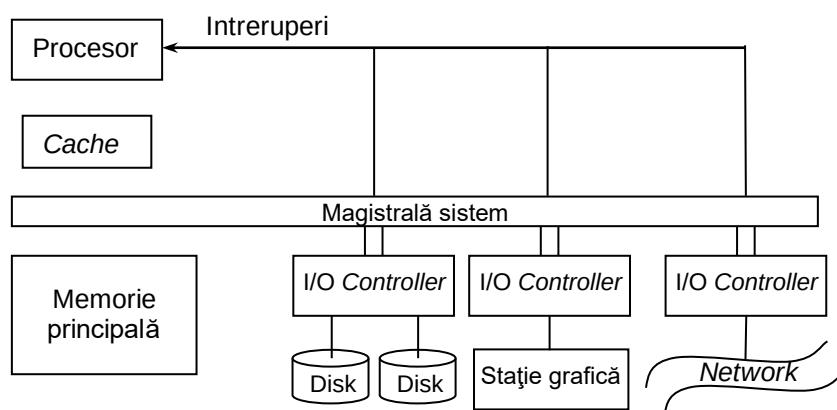
facilitează transferul de date și sincronizarea componentelor conectate la magistrala respectivă.

Caracterul de sistem deschis al calculatoarelor moderne este susținut în mare măsură de standardizarea magistrelor. Astfel, avem astăzi două clase importante de magistrale: (1) *magistrale de sistem* – dezvoltate pentru conectarea UC (unității centrale) la celelalte componente ale calculatorului; exemple: MultiBus, ISA, EISA, PCI și USB; (2) *magistrale specializate* – dezvoltate pentru a optimiza transferul de date către/dinspre un anumit periferic; exemple: VESA, SCSI, GPIB. În ultima perioadă se observă o extindere a standardelor de magistrală pentru a răspunde cerințelor de comunicare ale noilor generații de procesoare.

Ținând cont de componentele sistemelor de calcul introduse în paragrafele anterioare, putem spune că o magistrală conectează dispozitivele de intrare/ieșire la procesor și la memorii. Mai mult, o magistrală reprezintă calea de comunicare partajată care folosește un set de conductori pentru a inter-conecta subsistemele respective.

Magistrala reprezintă interconectarea electrică, fizică și logică, între periferice, procesoare și memorii și, deci, definește protocolul de comunicare la nivelul cel mai de jos (*lowest level*).

Un ansamblu tipic de periferice conectate la procesor și memorii se poate reprezenta ca în figura următoare.



Folosirea magistrelor de comunicare are în principal avantaje legate de flexibilitatea conectării componentelor. Astfel, pentru o schemă deja definită se pot adăuga cu ușurință noi tipuri de periferice și, mai mult, perifericele pot fi mutate de la un SC la altul, dacă acestea folosesc același tip de magistrală.

În cazul în care magistrala este suprasolicitată, apare dezavantajul major al comunicării *gâtuite* (engl., *bottlenecked*). Această situație poate fi evitată prin fixarea (și respectarea) unui număr maxim de intrări (periferice conectate) pe magistrală.

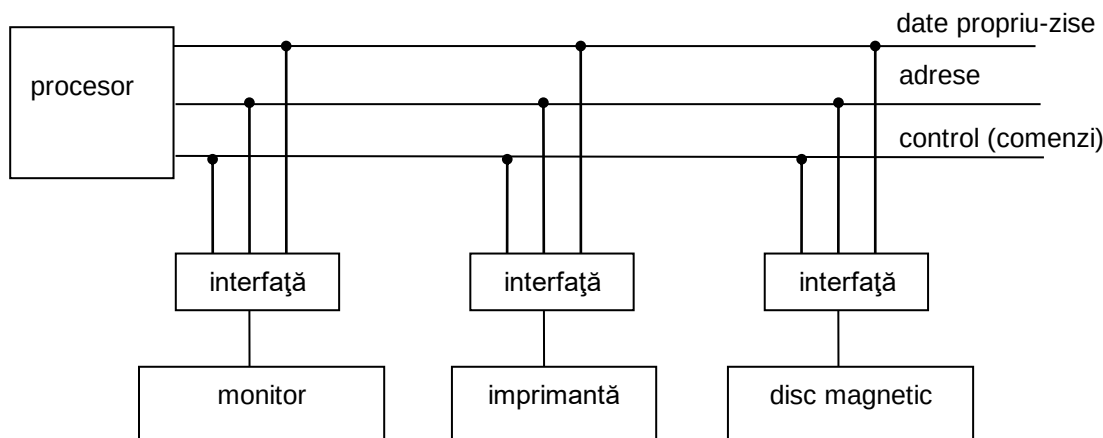
Performanțele unei magistrale se apreciază din două puncte de vedere fundamentale, anume viteza maximă de transfer a datelor pe acea magistrală și numărul maxim de periferice pe care le poate conecta. Scopul concret pentru care este construit fiecare model de magistrală va decide care dintre aceste criterii este prioritar și, în consecință, va valorifica la maximum acea caracteristică, în detrimentul celeilalte. Proiectarea unui model este dificilă, ținând cont și de faptul că viteza de

transfer este limitată la rândul ei de factori fizici, cum ar fi numărul de periferice conectate și lungimea magistralei.

Conform tipului de informații care circulă pe magistrală avem: magistrală de date, magistrală de adrese și magistrală de control (comenzi). Generic, ansamblul tuturor acestor linii este recunoscut ca **magistrală sistem**. Numărul liniilor de un anumit tip este proporțional cu viteza transferului pe liniile respective.

Pe liniile magistralei se transferă conținut de la o entitate sursă la o entitate destinație. Acest conținut poate fi date, comenzi sau adrese, în funcție de linia de magistrală pe care se transfera. Rolul liniilor/semnalelor de control este, de exemplu, să semnaleze cererile, să precizeze tipul datelor de pe liniile de date, să implementeze protocolul de magistrală. Deoarece magistrala este partajată (folosită în comun de mai multe entități), trebuie stabilit un protocol care să decidă univoc la fiecare moment cine folosește magistrala la momentul următor. Operația de bază cu care lucrează un protocol de magistrală este **tranzacția pe magistrală**, care constă, pe de o parte, din trimiterea adresei și din transmiterea și primirea datelor, pe de altă parte. O astfel de tranzacție este definită și de direcția de lucru cu memoria, rezultând **tranzacții de intrare și tranzacții de ieșire**.

Un **exemplu** de utilizare a unei magistrale sistem este cel descris în continuare. Dacă un disc vrea să scrie date în memorie de pe un anumit sector al său atunci liniile de adresă se vor folosi pentru a preciza adresa din memorie la care se va face scrierea datelor și pe liniile de date se vor transfera datele propriu-zise de pe disc. În acest caz, liniile de control vor indica codul comenzii propriu-zise de scriere pe disc.



Liniile de magistrală I/O care pornesc de la procesor sunt legate la unitățile de interfață ale tuturor perifericelor. Pentru a comunica cu un anumit periferic, procesorul pune pe liniile de adresă, adresa perifericului respectiv. (Ne amintim că portul este registrul de date propriu unei interfețe I/O. Aici se folosește tocmai această adresă a portului *din spatele* interfeței la care este conectat perifericul.) Fiecare interfață conectată la magistrala I/O conține un *decodificator de adrese* care decodifică conținutul de pe liniile de adresă. Când o interfață detectează propria sa adresă, deschide calea de comunicare (accesul între liniile de magistrală și perifericul pe care ea îl gestionează). Pentru toate interfețele cărora nu le corespunde adresa de pe magistrală, perifericele respective sunt inaccesibile.

Când procesorul pune adresa perifericului căutat pe liniile de adresă, în același timp furnizează pe liniile de control și codul unei funcții / comenzi (*function code*). Interfața selectată răspunde acestui cod și determină execuția funcției. Codul acesta este o **comandă de intrare/ieșire** și constă într-o instrucțiune de executat de către interfață pe perifericul asociat ei. Interpretarea comenzii depinde de perifericul căreia îi este adresată. O interfață de periferic poate primi următoarele patru tipuri de comenzi:

- 1) comenzi de control – care activează perifericul sau îl informează asupra următoarei operații;
- 2) comenzi de stare – cu rol în testarea stării interfeței și a perifericului. Erorile care apar la transferul datelor sunt semnalate prin valorile anumitor biți din registrul de stare. Procesorul citește periodic acest registru, detectând astfel situația în care a apărut o eroare;
- 3) comenzi de ieșire – care constau în comenzi de transfer al datelor înspre exterior, adică determină interfața să răspundă prin dirijare de date de pe magistrală într-unul din regiștrii săi de tranzit (*buffer register*);
- 4) comenzi de intrare – care sunt opuse comenzilor de ieșire. În acest caz, interfața primește un set de date de la periferic, pe care le depune într-unul din regiștrii săi de tranzit. Procesorul verifică printr-o comandă de stare dacă datele sunt accesibile și apoi transmite o comandă de intrare. Interfața depune datele pe liniile de date ale magistralei I/O de unde sunt preluate de procesor.

1.2.6.1. Clasificarea magistrelor

Din punctul de vedere al componentelor conectate, avem:

- 1) magistrale procesor - memorie;
- 2) magistrale de intrare - ieșire;
- 3) magistrale de extensie (*backplane*).

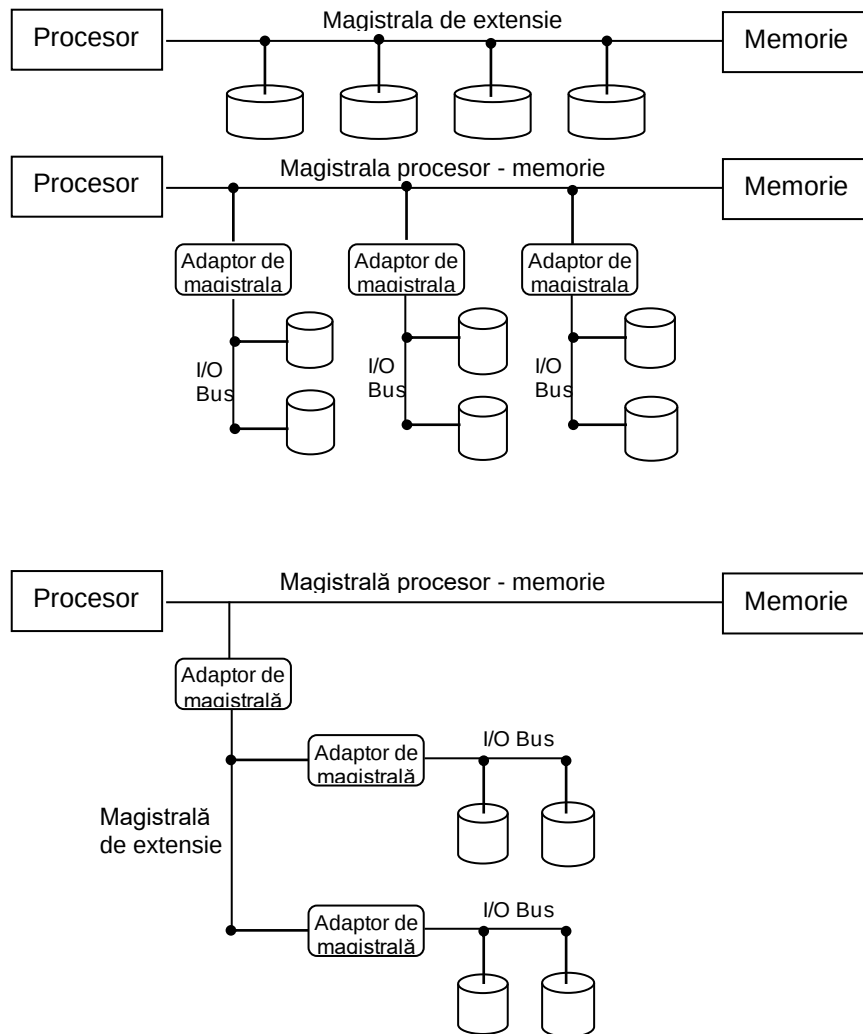
Magistralele procesor - memorie sunt, în general, scurte, dar de viteză mare și adaptate sistemului de memorie, astfel încât să maximizeze lungimea de bandă procesor - memorie și, implicit, cantitatea de informație transmisă simultan.

Magistralele de intrare/ieșire (I/O Bus) pot fi suficient de lungi, pot conecta mai multe tipuri de periferice și acceptă o varietate mare de lungimi de bandă pentru comunicarea cu perifericele conectate. Magistralele de intrare-ieșire nu interacționează direct cu memoria ci se folosește o magistrală procesor - memorie sau una de extensie pentru conexiunea cu memoria.

Magistralele de extensie (*backplane*) sunt construite pentru ca procesoarele, memoriile și perifericele să poată coexista pe aceeași magistrală. Astfel, o magistrală de extensie va răspunde atât cererilor de comunicare procesor - memorie, cât și cererilor de comunicare dintre periferice și memorie. Se numesc de extensie tocmai pentru că permit adăugarea în sistem a diferitelor plăci interschimbabile, prin inserarea acestor plăci în conectorii de extensie ai magistralei (sloturi).

Comparativ, putem spune că magistralele procesor - memorie au design specific și sunt, în general, monopol al firmei producătoare, în timp ce celelalte două tipuri (de intrare/ieșire și de extensie) pot fi refolosite în diferite sisteme și sunt construite pe baza unor **standarde de magistrală**.

În figurile următoare reprezentăm trei variante de interconectare a componentelor SC prin magistrale de comunicare.



Avantajele remarcabile ale variantei a treia, cea mai complexă, dar și cea mai recomandată în arhitecturile moderne, ar fi:

- magistrala procesor - memorie poate fi mult mai rapidă decât magistralele de extensie sau de intrare/ieșire;
- sistemul de intrare/ieșire poate fi completat cu multe *controller*-e sau chiar cu magistrale I/O pe magistrala de extensie – ceea ce nu afectează viteza magistralei procesor - memorie.

Din punctul de vedere al tipului de comunicare, avem:

- 1) magistrale sincrone;
- 2) magistrale asincrone.

O **magistrală sincronă** are liniile de control conectate / sincronizate cu ceasul / generatorul de tact al sistemului, care controlează un protocol bine definit pentru comunicare cu sincronizare. De obicei, magistralele procesor - memorie sunt magistrale sincrone, deoarece conectează doar puține componente și trebuie să lucreze la viteză mare de transfer a datelor.

Observație. Funcționarea unei magistrale sincrone poate fi simulată cu un automat finit determinist.

Prezența ceasului (a semnalului de tact) impune restricții în construirea și funcționarea unei magistrale sincrone. De exemplu, toate componentele conectate trebuie să se *sincronizeze* pe aceeași frecvență de tact. De asemenea, datorită problemelor de sincronizare, o magistrală sincronă nu poate fi și lungă și rapidă.

O **magistrală asincronă** nu este controlată de ceas, deci poate grupa o mai mare varietate de periferice și poate fi oricât de lungă. În acest caz, pentru coordonarea transmiterii de date se folosește un protocol de tip *hand shaking* (vezi și Moris Manno, pag. 393). Acesta constă dintr-o succesiune de pași în care sursa și destinatarul transferului de date trec la pasul următor numai *de comun acord* (*mutual agreement*). Acest protocol este practic implementat prin suplimentarea liniilor de control pe magistrală.

Observație. Funcționarea unei magistrale asincrone poate fi simulată cu o pereche de automate finit deterministe: o mașină trece în starea următoare numai dacă cealaltă a intrat într-o anumită stare - test.

1.2.6.2. Standarde de magistrală

Magistralele de intrare/ieșire servesc ca o modalitate de a îmbunătăți performanțele SC prin conectarea a noi periferice. Pentru a facilita această operație, industria sistemelor de calcul a dezvoltat o serie de standarde de magistrală.

Un **standard** reprezintă o specificare pentru un producător de componentă. Astfel, un **standard de magistrală** impune producătorilor structura de bază a magistralei, astfel încât să poată fi cunoscută și folosită a priori de producătorii perifericelor, cu compatibilitate maximă cu mașina gazdă.

Un standard uzual de magistrală este, de exemplu, **SCSI – Small Computer System Interface**. De obicei, o magistrală de acest tip este o interfață fizică către o magistrală de extensie sau către o magistrală procesor - memorie. Un *controller* SCSI coordonează transferurile de la periferic, pe magistrala de intrare - ieșire, spre memorie, prin intermediul unei magistrale procesor - memorie. Deci, magistralele de intrare - ieșire sunt construite conform acestui standard.

Pentru magistralele de extensie este recunoscut standardul **ISA – Industrial Standard Architecture**. Inițial au fost magistrale pe 8 biți și, ulterior, au fost extinse la 16 biți, ca standard pentru IBM PC/AT. Limitată la o frecvență de ceas de 8 MHz, această magistrală a fost deja insuficientă pentru procesoarele 80386. Totuși, o magistrală ISA (sloturi de 8 biți sau de 16 biți) rămâne o opțiune pentru conectarea imprimantelor prin porturi seriale, pentru *controller*-e de mouse, dar sunt prea lente pentru discurile moderne sau pentru adaptoare video.

Standardul **PCI – Peripheral Component Interconnect** – a fost inițiat de Intel și dezvoltat ulterior de alt concern industrial. Aceste magistrale sunt cele de mare viteză. PCI a fost prima magistrală care a răspuns cererilor de magistrală pe 32 biți reclamate de procesoarele Pentium și opera la frecvențe de tact de 33 MHz sau 66 MHz, permițând rate de transfer de 132 MB / sec sau 264 MB / sec. PCI suportă controlul total al magistralei (*bus mastering*). De obicei, magistralele de extensie sunt magistrale PCI. Arhitectura magistralei PCI poate oferi facilitatea *Plug-and-Play* (autoconfigurare). Această tehnologie constă în adaptarea automată a configurației calculatorului (fără intervenția utilizatorului) la caracteristicile componentei care

tocmai se instalează. Astfel, instalarea oricărui dispozitiv devine o operație simplă și sigură. În plus, autoconfigurarea face inutilă setarea manuală a configurației calculatorului. Sistemele de operare Windows moderne recunosc facilitatea *Plug-and-Play*.

Calculatoarele personale dispun de patru tipuri de arhitecturi de magistrală și anume: ISA, EISA (Extended ISA), Micro Channel și PCI. Magistrala PCI este adecvată pentru atașarea la calculator a perifericelor de viteză mare, dar este prea costisitor să existe câte o interfață PCI pentru fiecare periferic de viteză scăzută, de exemplu, tastatura, mouse-ul, camera foto/video digitală, scanner, telefon digital ș.a.m.d.. De aceea, pentru conectarea la calculator a dispozitivelor periferice de viteză redusă s-a impus un standard de magistrală unanim acceptat, anume **USB** (engl. *Universal Serial Bus*).

Rezumat.

Scopul acestui paragraf a fost prezentarea caracteristicilor esențiale ale principalelor periferice conectate la un calculator modern, adică periferice standard (tastatură, mouse, monitor) și periferice larg utilizate (discuri, imprimantă, modem, plăci de conexiune). Tratarea acestor periferice s-a făcut din mai multe puncte de vedere: (1) identificarea diferitelor tipuri de periferice (clasificări), (2) caracterizarea elementelor specifice fiecărui tip de periferic, (3) caracterizarea perifericelor din perspectiva conectării lor cu componentele cu care comunică direct prin magistrale.

Cuvinte cheie.

dispozitiv periferic, sistemul de intrare/ieșire
periferic de intrare, periferic de ieșire, periferic de intrare/ieșire
periferic de schimb, periferic de memorare
unitate de interfață, port
magistrală, standard de magistrală

Verificare.

1. Care sunt perifericele conectate la sistemul de calcul pe care îl folosiți în mod frecvent?
2. Explicați modul în care procesorul central stabilește comunicarea directă cu un anumit periferic solicitat la un moment dat.

NOTIȚE

1.3. Unitatea de memorie

Unitatea de memorie (*memorii*) este unitatea funcțională a calculatorului dată de totalitatea componentelor destinate memorării datelor și de circuitele asociate, necesare transferului de informații înspre și dinspre componentele care memorează / stochează / rețin conținut specific (date, informații, instrucțiuni).

1.3.1. Structura fizică a memoriei

Referitor la **structura fizică** a memoriei (modul de construire a componentelor), avem că, la primele calculatoare moderne, **memoria** a fost formată dintr-un mediu de memorare și sisteme de circuite electromagnetice pentru controlul gestiunii memoriei. Ulterior, din anii 1920, subsistemul de memorare este bazat mai ales pe memorii semiconductoare; concret, este alcătuit dintr-o succesiune de *celule binare* de tip condensatori sau **bistabili** / circuite (basculante) bistabile, ce permit sau nu trecerea curentului, fiecare stabilind astfel memorarea uneia dintre valorile 0 sau 1.

Observație. Reprezentarea a două stări posibile ale unui sistem prin 0 și 1 a determinat folosirea în sistemele de calcul în special a bazei de numerație 2 și a puterilor lui 2.

În general, cantitatea de informație care se câștigă prin precizarea stării unui sistem care are două stări posibile se numește **bit de informație**. În particular pentru memorii, prin această sintagmă vom înțelege *informația* referitoare la starea curentă a unei celule binare dată prin 0 sau 1, mai mult decât valoarea propriu-zisă a bitului menționată separat. Astfel, o celulă binară îndeplinește funcția de memorare a unui bit de informație. Conținutul unei celule binare este un **bit** (engl., *binary digit*).

În memorie, un grup de celule binare formează o **locație de memorie**. Pentru SC actuale, o locație are opt celule binare și se numește **octet (O)**. **Conținutul curent al unui octet este un byte (B)** sau **caracter**. Această echivalență de denumire provine de la faptul că un caracter din setul de bază se reprezintă în memorie pe un octet (vezi codul ASCII).

Octeții sunt numerotați de la 0 la capacitatea maximă a memoriei. Astfel, capacitatea memoriei este dată de cantitatea maximă de informație pe care aceasta o poate stoca, adică numărul de octeți dați eventual, multiplicativ: 1 KB (kilobyte) = $2^{10}B = 1024$ bytes, 1 MB (megabyte) = $2^{10} KB = 2^{20}$ bytes, 1GB (gigabyte) = $2^{10} MB = 2^{30}$ bytes ($2^{30} = (2^{10})^3 \approx (10^3)^3 = 10^9$), 1TB (terabyte) = $2^{10} GB = 2^{40}$ bytes. *Deoarece reprezentarea internă a datelor în memoria sistemelor de calcul este binară și nu în baza zece, producătorii de calculatoare și-au însușit aceste estimări binare ale prefixelor din Sistemul Internațional de Unități* [Wikipedia, *Since computer memory comes in multiples of 2 rather than 10, the industry used binary estimates of the SI-prefixed quantities.* <http://en.wikipedia.org/wiki/Byte>].

Numărul de ordine al unui octet în memorie este corelat cu **adresa de memorie** a grupului respectiv de celule binare. Această adresă joacă un rol esențial în regăsirea informației memorate și transmiterea ei către alte subsisteme. De exemplu, dacă procesorul cere transmiterea unei informații memorate atunci ea trebuie să furnizeze atât *adresa octetului de unde se face transferul*, cât și *numărul de octeți care se transferă*.

În terminologia modernă, numim **cuvânt** orice succesiune de 2, 4 sau 8 octeți. Astfel, calculatoarele din primele generații, până la 80286 inclusiv, au cuvântul format din 2 octeți, începând cu seria 80386, cuvântul este de 4 octeți, iar la microcalculatoarele moderne avem cuvântul de 32 biți = 4 octeți sau 64 biți = 8 octeți. Lungimea cuvântului este o caracteristică a procesorului, prin capacitatea registrului de bază al procesorului.

Rangurile celulelor binare ale unui cuvânt (implicit, biții unui byte) trebuie ordonate într-un mod unic. Pentru aceasta se folosește ordinea puterilor lui 2. Toate tipurile de SC numerotează biții începând cu 0. Dintre biții unui byte se remarcă:

- **bitul cel mai semnificativ, MSB**, cel care corespunde puterii celei mai mari a lui 2;
- **bitul cel mai puțin semnificativ, LSB**, cel care corespunde puterii celei mai mici a lui 2.

În funcție de modelul de procesor, rangul pentru biții unui byte se numerotează crescător de la stânga la dreapta (format *big endian*, specific procesoarelor Motorola) sau de la dreapta la stânga (format *little endian*, specific procesoarelor Intel), această a doua variantă fiind folosită uzual și de logica matematică și computațională.

1.3.2. Tehnologii de realizare a memoriilor. Memorii semiconductoare

Cronologic, principalele tehnologii de realizare a componentelor de memorare sunt: (1) tambur magnetic, (2) inele de ferită, (3) discuri magnetice, (4) memorii semiconductoare, (5) procedee optice.

Memoriile semiconductoare sunt circuite integrate bazate pe metale semiconductoare, de obicei Siliciu. Față de Germaniu, Siliciul este preferat deoarece își păstrează conductibilitatea chiar și la temperaturi înalte.

În sistemele de calcul binare, circuitele de memorie stochează informația binară în celule elementare de memorie cu capacitatea de 1 bit. Celulele de memorie sunt grupate în locații de memorie, care, în funcție de tipul de memorie, poate fi de 1, 2, 4 sau 8 celule binare. Impactul comercial pe care l-au avut procesoarele pe 8 biți în anii '80 a impus dimensiunea uzuală de 8 celule binare pentru o locație de memorie (un octet) și conținutul de memorie de un *byte* pentru un octet. Fiecărei astfel de locații îi este asociat un număr unic recunoscut ca **adresa** locației respective. Combinația binară citită / înscrisă într-o locație de memorie de un cuvânt (cuvânt de memorie) reprezintă conținutul locației respective și este un **cuvânt-memorie**. Numărul total de locații dintr-o componentă de memorare definește capacitatea circuitului de memorie respectiv. Intervalul (domeniul) adreselor definește **spațiul de adresare** al circuitului de memorie.

O memorie semiconductoare este caracterizată de următorii parametri:

- (1) geometria (sau modul de organizare a memoriei) – se referă la: modul de dispunere a celulelor bit, lungimea unui cuvânt de memorie, modul de adresare a cuvintelor;
- (2) capacitatea – exprimată în numărul de cuvinte – memorie; din punct de vedere practic, capacitatea unei memorii este dată de cantitatea maximă de informație care poate fi stocată la un moment dat pe memoria respectivă, măsurată în *bytes*;

- (3) latența sau timpul de acces – reprezintă diferența dintre momentul (final) în care memoria furnizează (pe liniile de ieșire) datele solicitate și momentul (inițial) în care memoria primește (pe liniile de intrare) adresa locației solicitate / locației care conține datele solicitate ($t_{\text{acces}} = t_{\text{DataOut}} - t_{\text{Adrin}}$); se măsoară în *ns* sau μs ;
- (4) ciclul memoriei – reprezintă timpul necesar pentru citirea/scrierea conținutului unei locații; cu alte cuvinte, ciclul memoriei este intervalul de timp dintre două aplicări succesive de adrese pe liniile de intrare în circuitul de memorie; se măsoară în *ns* sau μs ;
- (5) puterea consumată – se măsoară în $\mu W / bit$;
- (6) volatilitatea – o memorie este volatilă dacă își pierde conținutul în timp (memorii RAM dinamice, DRAM) sau la decuplarea de la sursa de alimentare electrică (memorii RAM statice, SRAM); memoriile de tip ROM sunt nevolatile;
- (7) tehnologia de realizare – din acest punct de vedere avem:
 - (a) memorii bipolare: foarte rapide, dar cu densitate mică de integrare; principalele tehnologii bipolare sunt: TTL (*Transistor-Transistor Logic*) și ECL (*Emitter-Coupled Logic*);
 - (b) memorii MOS (*Metal-Oxide Semiconductor*): cu densitate mare de integrare, putere consumată mică, dar cu timp mai mare de acces decât memoriile bipolare; memoriile de tip flash (*USB flash drive*) sunt realizate cu tehnologii de tip MOS.

Observație. Din definițiile de mai sus se observă că timpul de acces la memorie și ciclul memoriei caracterizează implicit și performanțele operaționale ale unei memorii (concret, viteza de execuție a operațiilor pe circuitul de memorie respectiv).

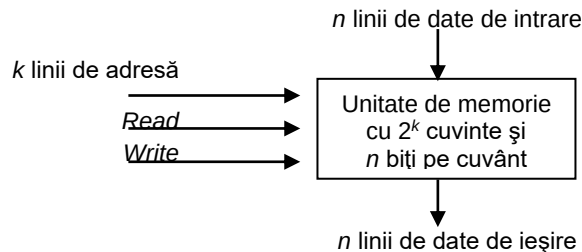
Principalele operații efectuate de circuitele de memorii semiconductoare și modelele corespunzătoare de memorii sunt:

- ✓ citire – memorii ROM (*Read Only Memory*);
- ✓ citire și scriere – memorii RAM (*Random Access Memory*); în particular, avem:
 - memorii RAM statice – mențin informația cât timp este menținută tensiunea de alimentare; uzual, celula de bit a unei memorii RAM statice este un bistabil;
 - memorii RAM dinamice – informația se pierde în timp, chiar dacă tensiunea de alimentare nu se întrerupe; uzual, celula de bit a unei memorii RAM dinamice este un condensator (nu un bistabil).
- ✓ programare – memorii PROM (*Programmable Read Only Memory*);
- ✓ regenerare – operație necesară, de exemplu în cazul memoriilor RAM dinamice; regenerarea este o citire incompletă (i.e. datele citite nu se utilizează în exteriorul circuitului ci, periodic, conținutul celulei de bit este selectat, citit și rescris intern în circuitul de memorie); uzual, rata de regenerare este de ordinul milisecundelor și nu depinde de capacitate.

Memoriile RAM sunt cu *acces direct*, deci informațiile pot fi accesate pentru transfer din orice zonă, aleasă aleator. De aceea, procesul de localizare a unei date în RAM este același pentru toate locațiile și cere un timp mediu egal, indiferent de

poziția fizică în memorie a locației respective. De aici provine numele de acces *aleator*. Astăzi, și memoria ROM este cu acces aleator la informație, astfel că acronimul RAM caracterizează mai corect memoriile de tip *Read/Write*.

O structură uzuală bloc de memorie RAM de tip $m \times n$ este un tablou bidimensional (matrice) de celule binare organizate în m cuvinte a câte n biți fiecare. Pentru $m=2^k$ avem un model standard de bloc de memorie RAM prezentat în figura următoare:



Comunicarea cu memoria RAM se face prin (a) linii de adresă, (b) linii de comandă (control) și (c) linii de date de intrare-ieșire.

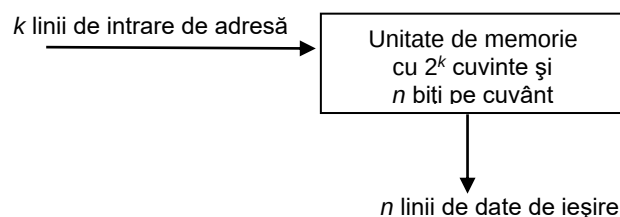
Cele k linii de adresă furnizează un număr binar pe k biți care adresează / indică adresa cuvântului care să fie ales pentru Read / Write, din cei 2^k existenți în memorie. Cele două intrări de control *Read* și *Write* dau direcția de transfer a conținutului cu memoria, prin faptul că ele precizează comanda, respectiv citește / preia conținut din memorie (*Read*) sau scrie / depune conținut în memorie (*Write*). Cele două seturi de câte n linii de date (n linii de intrare de date, respectiv n linii de ieșire de date) au rolul de a transporta datele propriu-zise în sensul respectiv.

În sistemul de calcul, o memorie de tip ROM este uzual folosită pentru memorarea programelor fixe (care nu se modifică în timpul funcționării SC) și a tabelor de constante care nu-și schimbă valoarea în sistem.

Într-o unitate de control, o memorie ROM poate fi folosită pentru conservarea informațiilor codificate care reprezintă secvențe de variabile interne de control necesare pentru activarea a diferite operații în sistem. O unitate de control care folosește ROM pentru memorarea informațiilor binare de control este numită *unitate de control cu microprogram (microprogrammed control unit)*.

Memoria ROM nu are nevoie de linii de control de tip *Read / Write* deoarece pe ROM avem doar citire de date; în fiecare moment cele n linii de ieșire furnizează automat cei n biți ai cuvântului selectat prin valoarea adresei de intrare.

O structură de memorie ROM de tip $m \times n$ este un tablou de celule binare organizate în m cuvinte a câte n biți fiecare. Pentru $m=2^k$ avem o memorie ROM de tipul:



Tehnologia circuitelor integrate s-a impus cu succes și în construcția memoriilor, astfel încât spunem despre un ansamblu de bistabili dispuși într-o ordine bine precizată că formează un **cip de memorie**. Cele mai răspândite modele de cipuri sunt de tip SIMM (engl. *Single Inline Memory Module*) sau DIMM (engl. *Dual Inline Memory Module*). O placă de memorie SIMM sau DIMM este un grup de cipuri de memorie cablate pe o placă de dimensiuni reduse și vândute ca o singură unitate (memorie). Deosebirea dintre SIMM și DIMM constă în felul în care este plasată seria de conectori (pini de contact) prin care memoria respectivă se conectează pe placa de bază și anume dispunere pe o singură parte sau pe ambele părți ale plăcii de memorie. Fiecare conector are o funcție bine delimitată în transferul de date memorate, în adresare sau în controlul memoriei respective.

1.3.3. Structura ierarhică de organizare a memoriei

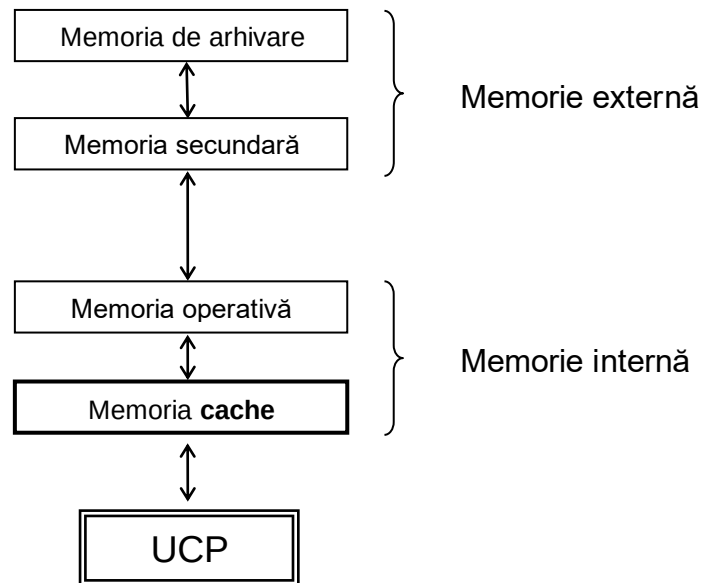
În timpul execuției lor, programele accesează în orice moment o zonă relativ mică a spațiului de adrese. Această observație a condus la delimitarea a două aspecte fundamentale în utilizarea memoriei, care sunt recunoscute ca **principiul localizării**. Astfel, avem **principiul localizării temporale** cu următorul enunț: *dacă este referită o locație de memorie, atunci este foarte probabil ca aceasta să fie referită din nou, în curând* și, respectiv, **principiul localizării spațiale**, cu următorul enunț: *dacă este referită o locație de memorie, atunci este foarte probabil ca locațiile din același spațiu de adresă să fie și ele referite*.

Aplicarea principiului localizării a condus la organizarea ierarhică a memoriei. Astfel, o **ierarhie de memorie** constă în nivele de memorii cu viteze de acces diferite și de dimensiuni diferite.

Din punctul de vedere al unui utilizator, memoria se împarte în *memorie externă* și *memorie internă*. *Memoria externă* este formată din toate componentele destinate stocării informației care sunt auxiliare sistemului de calcul. Cele mai folosite memorii externe sunt mediile de memorare bazate pe procedee optice (CD, DVD) și cele de tip disc (HDD, SSD). Toate celelalte componente de memorie ale sistemului de calcul formează *memoria internă*. Datele din memoria externă ajung să fie prelucrate de procesor numai după ce au fost transferate în memoria internă.

Din punctul de vedere al unui programator, informația stocată în memorie trebuie corelată cu programul curent de executat. Identificarea nivelelor de memorie trebuie să țină cont atât de *tipul conținutului memorat* (date propriu-zise sau programe), cât și de legăturile dintre informația stocată și programul curent în execuție la nivelul procesorului. În consecință, pentru un programator, memoria este organizată ca *memorie secundară* și *memorie principală*. *Memoria secundară* (externă sau auxiliară) este de capacitate mare, practic nelimitată, conține informația păstrată pentru o utilizare ulterioară și este formată concret din periferice specializate. *Memoria principală* (internă sau operativă) este de capacitate fixă și conține informația în curs de prelucrare, adică programele în curs de execuție și datele programului curent. Memoria principală este conectată direct la magistrala sistem pentru a permite o viteză mare de transfer al informației.

Ținând cont de considerentele anterioare, memoria se poate prezenta ierarhic, pe nivele succesive de descriere, așa cum rezultă din schema următoare:



Săgețile de pe schema de mai sus arată că toate componentele de memorie comunică între ele în ambele sensuri. În plus, *memoria cache* comunică direct cu procesorul.

Când o dată este solicitată de procesor, ea este căutată succesiv în zonele de memorie corespunzătoare nivelelor de mai sus: memorie cache, memorie operativă, memorie secundară, memorie de arhivare. Pentru regăsirea rapidă a datelor căutate, aceste memorii sunt construite astfel încât viteza de acces cea mai mare este la memoria cache și scade pentru celelalte tipuri, proporțional cu *distanța* față de procesor. În același timp, crește capacitatea de memorare, astfel că memoriile de arhivare ajung la capacități suficiente pentru stocarea de colecții mari și foarte mari de date.

În continuare ne vom referi pe rând la tipurile de memorie din ierarhia de mai sus.

Memoria de arhivare sau **memoria auxiliară** este dată de dispozitivele de memorare care rețin conținut pe termen lung, de exemplu, resurse informaționale sau programe între rulări. Acestea sunt gestionate de utilizator pentru folosirea eficientă a datelor depuse pe ele, de obicei sub formă de resurse informaționale (fișiere, foldere) sau ca și baze de date personale. Aceste suporturi de memorare sunt nevolatile, de obicei discuri magnetice (din 1965).

Ca entitate fizică a SC, memoria se mai numește **memorie reală**. Ca entitate în timpul utilizării ei, uneori avem nevoie și de termenul de memorie **virtuală**. Diferențele dintre acestea depind de modul de organizare fizică și de complexitatea memoriei în discuție.

Memoria virtuală este întotdeauna considerată ca o succesiune de octeți, în timp ce fizic, poziția a doi octeți vecini în memoria virtuală poate să difere. Legătura dintre un octet considerat în memoria virtuală și corespondentul său fizic este făcută de mecanismul de adresare.

Memoria secundară apare la sistemele care cunosc mecanismul de memorie virtuală. Aceasta reprezintă o **extensie** a memoriei operative. Începând cu 1985, după ce a apărut sistemul 286 și modul de lucru protejat, calculatoarele IBM-PC au

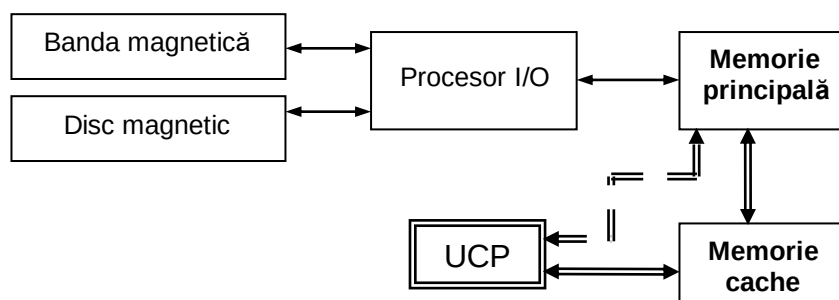
fost dotate cu **memorie extinsă** care permite o adresare naturală a spațiului de peste un megaoctet de memorie.

Memoria operativă conține datele și programele pentru toate procesele active la un moment dat în sistem. O dată cu încheierea unui proces, automat se eliberează zona din memoria operativă alocată procesului respectiv.

Memoria cache (în *engleză*, *franceză* ascunzătoare, depozit) este memorie internă ultra-rapidă, care comunică direct cu procesorul. Este de capacitate mică, dar cu viteză de acces foarte mare. În această zonă sunt reținute în fiecare moment datele cele mai recent utilizate de procesor. Data solicitată de procesor în prelucrarea curentă este adusă din zona de memorie în care a fost găsită și depusă în memoria cache *împreună cu* conținutul unui număr de locații vecine datei solicitate, astfel încât, împreună, să umple memoria cache. Această metodă este recunoscută ca **principiul vecinătății** și este justificat astfel: față de data curent solicitată, este foarte probabil ca următoarea dată solicitată de procesor să fie în memorie într-o locație apropiată de data curentă. O dată ce data curentă a fost adusă în cache împreună cu datele din locații alăturate, rezultă că următoarea regăsire se va face rapid, direct din zona cache. Acest principiu este strâns legat de **principiul localizării**.

În general, vom numi **memorie principală** acea zonă de memorie / locații care comunică direct cu procesorul. Ea va conține datele și programele utilizate de procesor la momentul curent (sunt în curs de utilizare). Memoria principală (primară) este volatilă, implementată cu memorii de tip DRAM.

Schema următoare precizează legăturile care se stabilesc între diferite componente ale sistemului de calcul implicate în mecanismul de transfer al datelor între procesor și memorii.



Din reprezentarea anterioară rezultă că dacă sistemul este recunoscut cu memorie *cache* atunci aceasta preia funcțiile memoriei principale, fiind direct conectată la procesor și realizând totodată organizarea transferului de informații între memoria principală și procesor.

Principalul rol al memoriei cache este de a compensa diferența între viteza de acces la memoria principală și viteza de prelucrare a procesorului. De obicei, procesorul este mai rapid decât memoria și, de aici, în absența mecanismelor de tip cache, activitatea procesorului poate fi încetinită nedorit.

De asemenea, se remarcă prezența procesorului de intrare-ieșire cu rolul de a gestiona transferul datelor între memoria auxiliară și memoria principală. Se observă că memoria auxiliară nu are acces direct la procesor.

Astăzi există trei tehnologii de bază folosite în construcția ierarhiilor de memorii:

1. memoria principală este implementată cu DRAM-uri – memorii de viteză medie și capacitate medie;
2. nivelele mai apropiate de UCP (memoriile cache) folosesc SRAM-uri (*Static RAM*) – memorii de viteză mare și capacitate mică;
3. nivelele de memorii lente, dar de capacitate mare folosesc discuri magnetice.

Memoriile de tip SRAM sunt ceva mai scumpe per bit decât memoriile de tip DRAM, sunt mult mai rapide și consumă mai puțină energie electrică.

Tehnologia de memorare	Timp de acces standard	\$ / MB (1997)
SRAM	5 ns – 25 ns	100\$ – 250\$
DRAM	60 ns – 120 ns	5\$ – 10\$
Disc magnetic	10 ms – 20 ms	0,1\$ – 0,2\$

Uzual, nivelele de memorii interne realizate cu structuri de tip RAM sunt numite *memorii RAM*. Astfel, spunem că *memoria RAM* este memoria internă în care se reține conținutul util programelor curente, în RAM se memorează cea mai mare parte a programelor, precum și datele care se modifică în timpul funcționării sistemului de calcul. Volatilitatea structurilor de tip RAM impune executarea de către programator a unei operații explicite de *salvare* a programului utilizator în curs de dezvoltare.

Între discurile magnetice și memoria principală există trei diferențe esențiale:

1. discurile sunt nevolatile – deoarece se bazează pe tehnologii de memorare prin magnetizare (polarizare);
2. discurile dure au acces lent – deoarece se bazează pe dispozitive mecanice;
3. discurile sunt mai ieftine pe Mega-Byte – deoarece au capacitate de memorare mult mai mare, la un preț rezonabil; în plus, prețul de cost al unei secțiuni de disc este mai mic decât al unui circuit integrat (care stă la baza construcției componentelor memoriei principale, DRAM-uri).

1.3.4. Memorii cache

La prima mașină comercială care avea un nivel intermediar de memorie între procesor și memoria principală, acest nivel s-a numit **cache**. Astăzi, deși acesta este înțelesul propriu al cuvântului *cache*, termenul este folosit pentru a ne referi și la ORICE tip de memorie gestionat astfel încât să optimizeze accesul la locația căutată. Deci, **cache** denumește o metodă de obținere a unui acces rapid la porțiunile de cod și de date cele mai recent utilizate.

Memoria cache este organizată în blocuri de lungime fixă, numite linii de cache. O astfel de linie este o copie a unei zone din memoria din care a fost adusă informația în cache.

Proiectarea unei memorii cache trebuie să răspundă la întrebări de tipul:

1. care este dimensiunea optimă a unei linii de cache?
2. cum se regăsește informația conținută în cache?

3. care linie se înlocuiește în cazul în care la un transfer memoria cache este plină?

În funcție de modurile de soluționare a acestor probleme, sunt recunoscute mai multe arhitecturi de cache, cum ar fi: memorii cache cu mapare directă, memorii cache asociative, memorii cache set asociative, memorii cache organizate pe sectoare.

1.3.5. Memoria virtuală

După cum memoria cache asigură accesul rapid la informațiile cel mai recent utilizate, *memoria principală poate acționa ca un cache* pentru memoria secundară. Această tehnică se numește **memorie virtuală**. Astfel, memoria virtuală este un mecanism folosit pentru a extinde limitele memoriei fizice. Într-un sistem cu memorie virtuală, aceasta apare utilizatorului ca și cum integral spațiul logic de adrese este disponibil pentru memorare. De fapt, în orice moment, numai câteva pagini din spațiul logic de adrese sunt mapate peste spațiul fizic. Alte pagini (nici) nu sunt prezente în memoria principală. În schimb, informația din aceste pagini este memorată temporar într-o memorie secundară de tip disc (al cărui cost pe bit este mult mai scăzut). Ori de câte ori este accesată o pagină care lipsește, sistemul de operare încarcă pagina respectivă de pe disc și memorează pe disc o altă pagină, care nu a fost recent referită. Prin acest mecanism, utilizatorul are impresia unei memorii fizice uriașe, dar mai lente.

Există două funcții majore pentru memoria virtuală:

- 1) să permită partajarea eficientă și sigură a memoriei între mai multe programe;
- 2) să permită unui program utilizator unic să mărească logic dimensiunile memoriei primare (principale).

1) Pentru justificarea primei motivații considerăm un set de programe care se execută simultan pe o mașină. Memoria totală solicitată de toate aceste programe poate fi mult mai mare decât cantitatea de memorie principală disponibilă pe mașină, dar, numai o parte din această memorie este EFECTIV folosită la un moment dat. Memoria principală are nevoie numai de *secvențele active* (procesele curente) ale programelor principale — la fel cum în cache se găsește la un moment dat numai secvența activă (procesul curent) ale UNUI program. Evident, pentru a permite mai multor programe să partajeze aceeași memorie, trebuie ca programele să fi fost protejate unele de altele (un program poate să scrie sau să citească NUMAI zona de memorie care i-a fost asigurată).

De exemplu, la compilarea simultană a mai multor programe nu se poate ști care programe partajează memoria în fiecare moment. De fapt, programele care partajează se modifică dinamic (în timpul execuției). Datorită acestei interacțiuni dinamice trebuie să se compileze fiecare program în propriul lui *spațiu de adresă*. Acesta va fi o secvență de zone de memorie separată, accesibilă numai programului respectiv.

Mecanismul de adresare asociat memoriei virtuale asigură traducerea spațiului de adresă al unui program în adrese fizice. Acest proces de traducere impune protecția spațiilor de adrese între ele.

2) Pentru justificarea celei de-a doua motivații a utilității memoriei virtuale considerăm următoarea situație. Formal, dacă un program a devenit prea mare pentru memorie atunci este anunțat programatorul. Pentru a determina cauza

necesarului mare de memorie, programatorul împarte programul în secvențe și identifică porțiunile de cod care se exclud reciproc. Aceste depășiri (*overlays*) sunt sau nu încărcate, sub controlul programului utilizator. Depășirile de acest tip se organizează în module, fiecare conținând atât date cât și cod. Apelurile între procedurile aflate în module diferite pot sugera programatorului încapsularea modulelor.

Toate aceste responsabilități sunt preluate din grija programatorului de către memoria virtuală, care va gestiona automat nivelele de memorie principală (DRAM-uri) și memorie secundară (hard-discuri) din structura ierarhică de organizare a memoriei.

În contextul memoriei virtuale, memoria principală se mai numește și **memorie fizică**. O adresă din memoria principală este o **adresă fizică**. Mulțimea acestor adrese se va numi **spațiu de adrese**. În contextul memoriei virtuale, UCP lucrează cu **adrese virtuale**, care sunt translate (printr-un mecanism combinat hard și soft) în adrese fizice.

O **adresă logică** este o adresă utilizată de un programator într-o instrucțiune. Adresele care pot fi utilizate de un program constituie spațiul de adrese logice. Organizarea acestui spațiu de adrese definește arhitectura memoriei.

În plus, **memoria virtuală** simplifică tehnica de încărcare a programelor pentru execuție prin introducerea relocării. **Relocarea** aranjează adresele virtuale folosite de un program la diferite adrese fizice, ÎNAINTE ca adresele să fie folosite pentru a accesa memoria. Acest mecanism de relocare permite încărcarea programului în orice locație a memoriei principale cu același consum de resurse. Astfel, memoria virtuală reprezintă un concept arhitectural de organizare a memoriei prin care memoria operativă este extinsă pe spațiul de adresare al memoriei externe.

Există două tehnici independente de implementare a memoriei virtuale: segmentarea și paginarea. Cele două tehnici pot fi combinate în scopul extinderii virtuale a spațiului de adrese și pentru un control mai eficient al alocării spațiului de memorare. Ambele metode presupun divizarea spațiului de memorie (internă și externă) în blocuri. La execuția unui program se încarcă în memoria internă operativă numai acele blocuri care sunt necesare pentru execuție. Astfel, aplicarea tehnicilor de implementare a conceptului de memorie virtuală determină creșterea resurselor de memorare disponibile pentru programul utilizator și facilitează protejarea zonelor de memorie alocate modulelor de program. O eficiență crescută a utilizării memoriei virtuale se remarcă în sistemele de operare *multi-tasking* și *multi-users*.

Segmentarea presupune divizarea memoriei în zone compacte numite segmente. Acestea pot să aibă lungime fixă sau variabilă. Ele pot să ocupe intervale disjuncte în spațiul de adrese sau pot să se suprapună total sau parțial. Adresarea unei locații de memorie se face prin precizarea segmentului și a adresei relative în cadrul segmentului respectiv. Un segment este caracterizat de: adresa de început a sa, lungime, drepturi de acces. Încărcarea segmentelor în memorie și amplasarea acestora în spațiul de adrese disponibil sunt sarcini ale sistemului de operare. Locul fizic în care se află o anumită variabilă sau secvență de program este în general transparent pentru programator. Implementarea efectivă a segmentării depinde de suportul hardware oferit de procesor. De exemplu, la modelele moderne de procesoare Intel segmentele logice sunt definite prin descriptori sau direct de programator, la nivel de cod sursă în limbaj de asamblare.

Paginarea presupune divizarea memoriei în zone de lungime fixă numite pagini. Paginile sunt încărcate în memoria internă operativă numai dacă sunt solicitate informații conținute în acestea. Paginarea este gestionată de sistemul de operare și este transparentă pentru utilizator.

Cele două tehnici, segmentarea și paginarea, pot fi combinate în scopul extinderii virtuale a spațiului de adrese și pentru un control mai eficient al alocării spațiului de memorare. Ambele metode presupun divizarea spațiului de memorie (internă și externă) în blocuri. La execuția unui program se încarcă în memoria internă operativă numai acele blocuri care sunt necesare pentru execuție. Astfel, aplicarea tehnicilor de implementare a conceptului de memorie virtuală determină creșterea resurselor de memorare disponibile pentru programul utilizator și facilitează protejarea zonelor de memorie alocate modulelor de program. O eficiență crescută a utilizării memoriei virtuale se remarcă în sistemele de operare *multi-tasking* și *multi-users*.

În general, tehnicile prin care se implementează memoria cache și memoria virtuală sunt transparente pentru programele utilizator. Programele de aplicație sunt scrise ca și când ar lucra numai cu memoria principală.

Statistic, 90% din timpul rezervat pentru un program este utilizat pentru execuția numai a 10% din codul sursă al programului respectiv...

Rezumat.

Totalitatea componentelor destinate stocării datelor formează unitatea de memorie a sistemului de calcul. Acest paragraf detaliază aspecte concrete legate de structura și organizarea memoriei calculatoarelor moderne. Astfel, sunt abordate diferite puncte de vedere: (1) structura fizică a memoriei, (2) structura ierarhică de organizare, (3) structura internă dependentă de tipologia memoriei.

Cuvinte cheie.

bit, celulă binară, bit de informație
locație de memorie – conținut de memorie, octet – byte, cuvânt de memorie – cuvânt-memorie
principiul localizării, principiul vecinătății, cache
memorie cu acces aleator, memorie read/write

Verificare.

1. Calculați capacitatea totală de memorare (în megabytes) a unei dischete cu două fețe și care este formatată cu următoarele constante: 80 piste (cilindri), 18 sectoare, 524 octeți pe bloc (un bloc constă într-un sector de pe o pistă). Capacitatea unui bloc este dată de 512 octeți de date propriu-zise și 12 octeți auxiliari (necesari marcării începutului blocului și localizării acestuia).
2. Uzual, o pagină de text are 40 linii cu 75 de caractere pe linie. Câte pagini de astfel de text se pot memora pe un CD de 600MB?
3. Folosiți principiile de organizare a memoriei cache pentru a justifica caracteristicile acesteia.
4. Care este capacitatea unei memorii cu 12 linii de intrare de adresă și 16 linii de ieșire (intrare) pentru date?

NOTIȚE

1.4. Proiectarea calculatoarelor moderne

După Patterson și Hennessy [5], se pot enunța patru principii de bază ale proiectării tehnologiei hardware, pe care le enumerăm în continuare.

Principiul 1. Simplitatea favorizează uniformitatea [5, pag.97].

Principiul 2. Mai mic înseamnă mai rapid [5, pag.99].

Principiul 3. Proiectarea bună necesită compromisuri bune [5, pag.108].

Principiul 4. Cazul frecvent trebuie făcut să aibă execuție rapidă [5, pag.133].

Temă de reflecție.

Comentați aceste principii în contextul arhitecturii calculatoarelor. Cum regăsiți aceste principii în viața economică, socială, ... personală...?

NOTIȚE